

Trie, radix trie: Efficient string search

This chapter covers

- Understanding why working with strings is different
- Introducing trie¹ for efficient string search and indexing
- Introducing radix tree² as a memory-efficient evolution of trie
- Using prefix trees to solve string-related problems
- Leveraging tries to implement an efficient spell-checker

How many times have you sent a text or an email or tweeted in a hurry, only to realize a second later that you made a typo? For me, it was too many times! Lately, however, we've had a precious ally on our side in email clients and browsers in general: spell-checkers! If you'd like to know more about how they work and how to implement them efficiently, this chapter is the right place to start.

¹ Also known as prefix tree.

² Also known as radix trie or compact prefix tree.

In chapter 3 we described balanced trees, which offer the best compromise when it comes to containers and are ideal for efficiently storing dynamically changing data on which we need to perform frequent searches. In appendix C we discussed and compared the options we have for containers providing fast lookup, fast insertion, or fast removal, and trees offer the best tradeoff between all the operations.

Balanced trees, in particular, guarantee logarithmic running time in the worst-case for all the main operations. In the general case, when we don't know anything about the data we need to store and (later) search, this is really the best we can hope.

But what happens when we know that we will only store certain types of data in a container? Turns out there are several cases where if we have more information on the kind of data we need to handle, we can leverage better algorithms than the general-purpose ones.

Take, for instance, sorting: if we know that keys are integers in a limited range, we can use *RadixSort*, which means achieving better-than-linearithmic performance, defying the lower bound for sorting by comparison.³

Likewise, if we know that we need to sort strings, there are several specialized algorithms, such as the *3-way string quicksort*, that are optimized for this kind of data and perform better than plain, general-purpose quicksort (or any comparison-based sorting algorithm).

In this chapter, we will analyze a particular sub-class of containers, string containers, and investigate how we can optimize them both with respect to memory and running time by introducing new, specialized data structures: tries and radix tries. Then we'll use those containers to implement an efficient spell-checker.

6.1 *Spell-check*

But first, let's introduce the problem that we will solve in this chapter: spell-checking.

Not that the problem really needs any introduction, right? We would like to have a piece of software that can take words as inputs and return `true` or `false`, respectively, whether the input is a valid English word⁴ or not.

That's a bit of a vague description that leaves the door open to many possible (and possibly inefficient) solutions; we need more context to clarify our requirements. Let's assume that we are developing a new client for a social network on a client with low resources (for instance, on a mobile OS) and we need to add a live spell-checker that produces the classic red wavy underline below a misspelled word. Every time we type a word (and every time we type a word separator, like a space, commas, and so on) we need to check whether there is a typo.

³ It has been proven that it's not possible to sort a list of n elements with less than $O(n \log n)$ operations if the method used is exclusively based on comparisons. *RadixSort* can get as low as $O(n \log k)$ for sorting n integers that can take any of k possible values. For large values of k , when $k \sim n$, the algorithm behaves no better than the generic linearithmic bound.

⁴ Obviously, it's possible to write spell checkers for any language; choosing English is just a matter of convenience.

Due to scarce resources, we need our spell-checker to be fast and lightweight; we need to reduce its impact as much as possible, both in terms of CPU and memory usage.

But we would also like our spell-checker to be able to learn—for instance, so users can add their names or the name of their town/club/favorite artists and so on.

6.1.1 *A prncess, a Damon, and an elf walkz into a bar*

Have you noticed the typos in this section's title?⁵ Typos are always around the corner when you need to send a message in a hurry (regardless of the medium used) and unfortunately, once these messages are sent out, there is no going back—you can't edit them anymore.⁶

That's why a spell-checker that clearly highlights typos comes in handy, and today it's often included natively in browsers.

In the end, the design of our spell-checker is pretty simple: it is a wrapper around a *dictionary*,⁷ and the client's method that checks spelling just calls the `contains` method, and if the result is a miss, adds the visual feedback to show the error.

In turn, the design of our container's API is also simple. It's a generic container supporting search, like the API of binary search trees or Randomized Treaps (described in section 3.4).

We can already think about how to implement this container with the tools that we learned. For instance, if we knew that we had to support fast lookups on a static set, then we would have chosen a hash table or, if we could trade saving some memory for a certain loss of precision, we could even resort to a Bloom filter. Since the requirement is maintaining an open, dynamic set, however, the one data structure providing the best compromise for all the operations would be a tree.

A simple binary tree could, of course, be enough to support all the operations provided by dictionaries. Figure 6.1 shows a possible representation of what these trees could look like, where we chose to show only a small part of the subtree containing a few similar words (we'll see why this is relevant in a few lines).

How fast can operations on such a tree be? Assuming the tree is balanced, its height will be logarithmic in the number of words it contains, so for each call to `contains`, `insert`, `remove`, and so on, we'll need to traverse on average (and at most) $O(\log(n))$ nodes.

So far, however, when we've analyzed trees, we've assumed their keys were either integers or could be checked in constant time and require a constant space.

⁵ Ring a bell? It should . . .

⁶ With tweets, of course, you can now delete them—if you are fast enough—even before someone notices.

⁷ Here the term refers to the abstract data structure called a dictionary that, incidentally but not surprisingly, is used to model the digital equivalent of actual dictionaries. See appendix C and chapter 4 for more details.

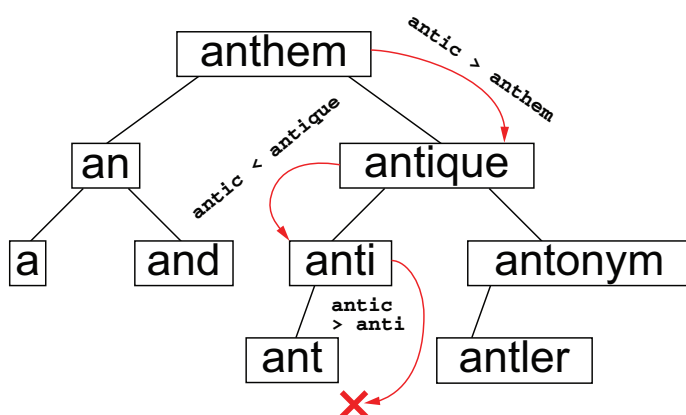


Figure 6.1 A BST storing (part of) a dictionary, and the steps to search the word “antic”. In this example, it’s a miss because the word was not stored in the tree shown.

For generic strings, this assumption is not realistic anymore. Each node will need to store a string of unbounded length, so the total memory needed to store the tree will be the sum, for each node, of all the keys’ lengths:

$$E[S(n)] = E\left[\sum_{i=0}^{n-1} |k_i|\right] = \sum_{i=0}^{n-1} E[k_i] \approx \sum_{i=0}^{n-1} L = n \cdot L$$

If we assume that the average length of the strings held by the tree is L , the expected value for $S(n)$, the space needed to store the tree, is proportional to $n \cdot L$. If the maximum length for strings is denoted with m , then $S(n) = O(n \cdot m)$ is a strict upper bound for the worst case.

Likewise, if we look at the running time of the search method, we see that we can’t ignore the strings’ lengths anymore. For instance, a call to `search(“antic”)` on the tree in figure 6.1 would start from the root and compare “antic” to “anthem”, which would require at least four characters to be compared before verifying the two words are not the same. Then we would move to the right branch and again compare the two strings, “antic” and “antique” (five more character-to-character comparisons), and since they don’t match, traverse the left subtree now, and so on.

Therefore, a call to search would require, worst case, $T(n) = O(\log(n) \cdot m)$ comparisons.

6.1.2 Compression is the key

This quick analysis shows that using a tree is not ideal, either space-wise or performance-wise. If we look more closely at the tree in figure 6.1, we can see that there seems to be a lot of overhead: all words start with the character “a”, but this is stored once for every node of the subtree in the illustration, and for each step in the traversal of the tree, it will be compared with the text that is being searched (or inserted).

Looking at the path to search for the word “antic”, all the four nodes shown and traversed share the same prefix, “ant”. Wouldn’t it be nice if we could somehow compress these nodes and only store the common prefix once with the deltas for each node?

6.1.3 Description and API

The data structure that we are going to introduce in the next section was created to answer these needs, and also to offer an efficient way to solve another operation: find all the keys in the container that start with the same prefix.

From our example in the previous sub-section, you can already see that if we were able to somehow store the common prefixes of strings only once, we should be able to quickly access all strings starting with those prefixes.

Table 6.1 shows the public API for an abstract data structure (ADT) that supports the usual container basic operations, plus two new ones: retrieving all the strings starting with a certain prefix and finding the longest prefix of a string stored in the container.

From what we saw in the previous example, `PrefixTree` could be a good name for this data structure, although `StringContainer` is more generic (as an abstract data structure, we don't care if it's implemented using a tree or some other concrete counterpart.) It conveys the gist of this container: being specific for strings. The fact that prefix search is supported is almost a natural consequence of designing a container for strings.

Table 6.1 API and contract for `StringContainer`

Abstract Data Structure: <code>StringContainer</code>	
API	<pre>class StringContainer { insert(key) remove(key) contains(key) longestPrefix(key) keysStartingWith(prefix) }</pre>
Contract with client	Besides all the operations of a regular, plain container, this structure allows us to search for the longest prefix of a string that is stored in it and return all the stored strings that start with a certain prefix.

Now that we have fixed an API and described the ADT that we will use to solve our spell-check problem, we are ready to delve into more details and see a few concrete data structures that could implement this ADT.

6.2 Trie

The first implementation of `StringContainer` that we will illustrate is the *trie*; incidentally, all the other data structures that we will show in the next sections are based on tries, so we couldn't choose to start anywhere else.

The first thing you should know about trie is that it's actually pronounced "try." Its author,⁸ René de la Briandais, chose this term because it was similar to tree, but also

⁸ De La Briandais, Rene. "File searching using variable length keys." Papers presented at the March 3-5, 1959, western joint computer conference. ACM, 1959.

because it's part of the word *retrieval*, which is the main purpose of this container; its peculiar pronunciation is partly meant as an indented pun, and partly meant to avoid confusion with "tree."

Tries were originally developed as a compact, efficient way to search strings in files; the idea behind this data structure is, as we saw in the previous section, providing a way to reduce redundancy by storing common prefixes of strings just once.

This couldn't be achieved using a plain binary search tree, or with just a binary tree, so a paradigm shift was needed: de la Briandais then used *n*-ary trees, where edges are marked with all the characters in an alphabet, and nodes just connect paths.

Nodes also have a small but crucial function: they store a tiny bit of information stating if the path from root to current node corresponds to a key stored in the trie.

Let's take a look at figure 6.2 before moving to a more formal description. It shows the structure of a typical trie, containing the words "a", "an", "at", and "I".

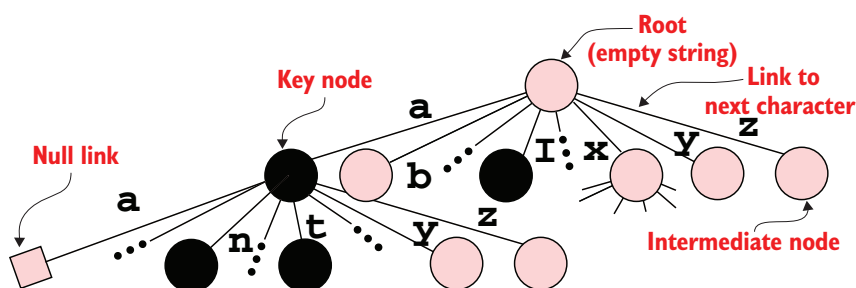


Figure 6.2 The structure of a trie. Words are encoded in the tree using edges, each edge corresponds to a single character, and each node *n* is associated with a single word, the one obtained by joining the characters associated with the edges in the path from the root to *n*. The root node corresponds to the empty string (because no edge is traversed), the leftmost leaf corresponds to "aa", and so on. Not all the paths make meaningful words, and not all the nodes store the word associated with them. Only filled, black nodes (called "key nodes") mark words stored in the trie, while hollow nodes, aka "intermediate nodes," correspond to prefixes of words stored in the trie. Notice that all leaves should be key nodes.

If you feel that figure 6.2 is a bit confusing, you are right. In their classic implementation, a trie's nodes have one edge for each possible character in the alphabet used: some of these edges point to other nodes, but most of them (especially in the lower levels of the tree) are null references.⁹

Nevertheless, this specific representation of tries looks terrible on paper: too many links and too many nodes that result in chaos.

That's why we will use an alternative representation, shown in figure 6.3. We'll only show links to actual trie nodes, omitting links to null.

⁹ As we'll shortly see, this happens for all characters *c* for which there is no suffix of the current node whose next character is *c*.

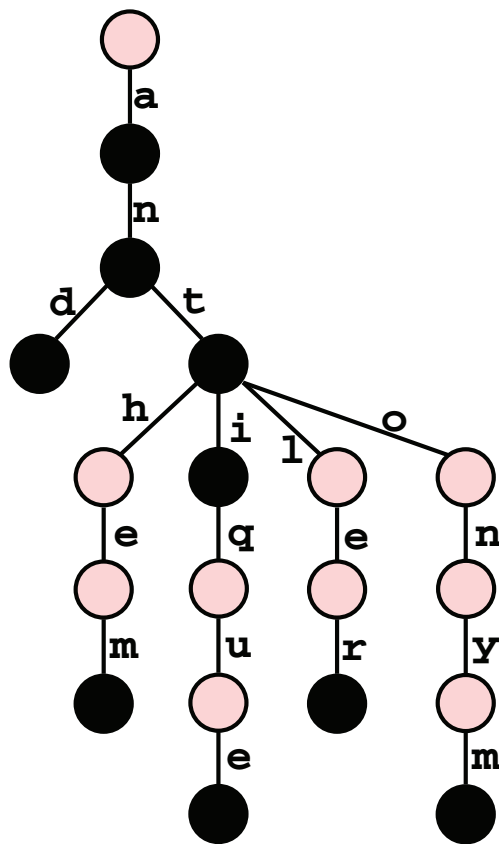


Figure 6.3 A more compact representation of a trie. This example trie contains the same elements as the binary search tree in figure 6.1.

Formally, given an alphabet Σ with $|\Sigma|=k$ symbols, a trie is a k -ary¹⁰ tree where each node has (at most) k children, each one marked with a different character in Σ ; links to children can point to another node, or to `null`.

Unlike k -ary search trees, though, no node in the tree actually stores the key associated with it. Instead, the characters held in the trie are actually stored in the edges between a node and its children.

Listing 6.1 illustrates a possible implementation of a `Trie` class (in object-oriented pseudo-code); you can also take a look at a full implementation on the book's repo on GitHub.¹¹

Listing 6.1 Class `Trie`

```
#type Char[]
Alphabet

class Node
```

¹⁰Although we'd usually talk about n -ary trees, we also usually reserve n to denote the number of entries in a container (or, in general, the input size for a problem). To avoid confusion, then, we will use k for the size of the alphabet and, consequently, the term k -ary.

¹¹See <https://github.com/mlarocca/AlgorithmsAndDataStructuresInAction#trie>.

```

#type boolean
keyNode

#type HashMap<Char, Node>
children

function Node(storesKey)
  for char in Alphabet do
    this.children[char] ← null
  this.keyNode ← storesKey

class Trie
  #type Node
  root

  function Trie()
    root ← new Node(false)

```

In the simplest version, trie’s nodes can only hold a tiny piece of information, just true or false. When marked with true, a node *N* is called a *key node* because it means that the sequence of characters in the edges of the path from the root to *N* corresponds to a word that is actually stored in the trie. Nodes holding false are called *intermediate nodes* because they correspond to intermediate characters of one or more words stored in the trie.

As you can see, tries go beyond the usual duality between leaves and inner nodes, introducing another (orthogonal) distinction. It turns out, though, that all leaves in a well-formed, minimal trie are key nodes: a leaf would make no sense as an intermediate node, as we will see.

The fact that words are stored in paths means that all the descendants of a node share a common prefix: the path from the root to their common parent. For instance, if we look at figure 6.3, we can see that all nodes share the prefix “an”, and all but one node share “ant”. These two words, incidentally, are also stored in the trie, because the nodes at the end of their path are key nodes.

The root is, to all extents, an intermediate node associated with the empty string; it would be a key node only if the empty string belonged to the corpus contained in the trie.

While for spell checkers storing a Boolean in each node could be enough (after all, we only need to know whether a word is in a dictionary), tries are often used to store or index words in texts. If that’s the case, we often need to know either how many occurrences of a word appear in the text or the positions where they appear. In the former situation we can store a counter in each node and only key nodes will have a positive value. In the latter we will instead use a list of positions, storing the index in the text where each occurrence starts.

6.2.1 *Why is it better again?*

Let’s address space first: why is the trie in figure 6.3 better than the binary search tree in figure 6.1?

Let's do some quick math! But first, we need to make some assumptions:

- We only consider ASCII strings and characters, so we have to account for 1 byte for each char (Unicode wouldn't change much; rather, it would make the savings obtained by using the cheapest option even greater) plus, in BSTs, 1 byte for each string's terminator.
- We only explicitly store links to actual nodes in tries and account for a fixed number of bytes for null-pointers in BSTs (the same space taken by non-null references).
- As we mentioned, each node in a trie has $|\Sigma|$ links, where $|\Sigma|$ is the alphabet size. This means that in tries, and especially in nodes in the lower levels, most links are `null`, and indeed in listing 6.1 it's also possible to see how all those links are initialized in `Node`'s constructor.
- For each node in a trie, we account for a fixed amount of space to store the children list (we can imagine that we use a hash table to store the link to children), plus a variable amount depending on the number of actual children.
- Each link in a tree will require 8 bytes (64 bit references), and each link in a trie will require 9 bytes (8 for the reference plus 1 for the character to which it's associated).
- Each node in the BST will require as many bytes as the number of characters in its key, plus 4 bytes¹² for the `Node` object itself.
- Each node in the trie requires 1 bit (to hold the Boolean) plus the same constant amount as for the BST; let's round up to 5 bytes.

Given these premises, we note that the BST in figure 6.1 has 9 nodes (whose keys are strings) and consequently $2 \times 9 = 18$ links, while the trie in figure 6.3 has 19 nodes and 18 links. For the BST, the root node contains the key "anthem" and requires 27 bytes (4 for the `Node` itself, 7 for the string, 2×8 for the links). Likewise, its left child, with key "an", requires 23 bytes. You see how it's computed; it's 21 bytes per node, plus the length of the string. For the whole tree, considering it has 9 nodes and requires a total of 47 bytes for the keys, we need 227 bytes.

Let's now check the trie: each node requires 5 bytes, and each link 9 bytes—a total of 257 bytes.

So, in practice, this trie might require a little more memory than the corresponding BST. These quantities depend on many factors; first of all, the overhead for objects. Since this trie has more nodes, the more this overhead is, the larger the delta will be.

Obviously, however, the shape of the tree and the actual number of nodes also play a big role. In the example in figure 6.3, only a short prefix is shared among the keys. It turns out that tries are more efficient when holding keys with a large shared prefix. The example in figure 6.4 shows how the balance can be in favor of tries in these cases. As

¹²This quantity is completely arbitrary; real objects in programming languages have an overhead, which can be way larger than 4 bytes (for instance, usually between 8 and 16 bytes in Java or C++).

you can see, in figure 6.4 most trie nodes are black (key nodes), and when the ratio of key nodes versus intermediate nodes is higher, intuitively it means that the efficiency of the trie is also higher, because when a path has more than one key node, we are storing at least two words in a single path (one of which is a prefix of the other). In a BST they would require two BST nodes storing both of the strings separately.

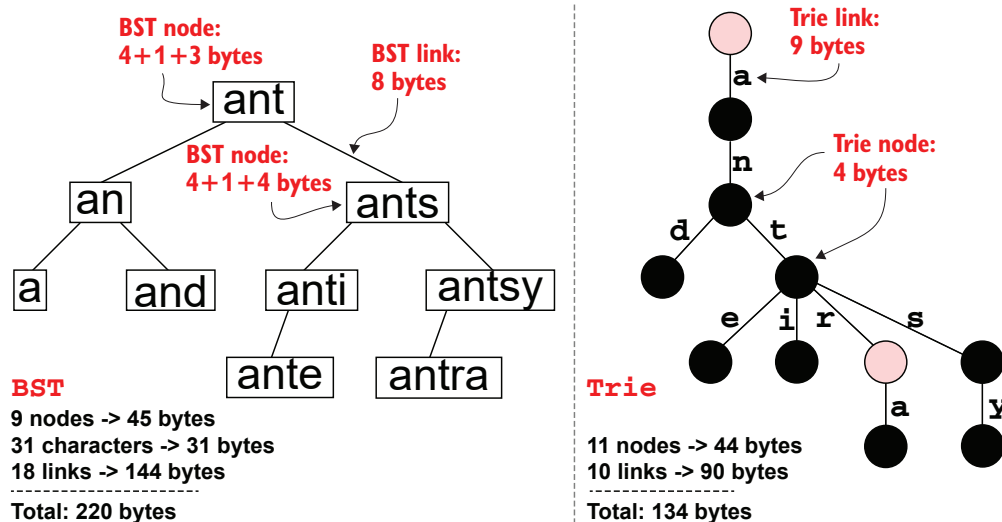


Figure 6.4 Comparing BST and trie approaches on a different example. When the keys in the trie have a larger ratio of shared characters (that is, a longer common prefix), tries are more efficient in storing strings

Another sign of more-efficient storage is when there are deep nodes branching out. In that case, the trie is “compressing” the space needed for two strings with a common prefix by storing the common prefix only once.

In the end, with just nine words stored in this second example, using the same assumptions as shown previously, the difference becomes 220 bytes versus 134 bytes, with tries saving almost 40% of the space. If we consider an 8-byte overhead for nodes, the difference would be 256 versus 178 bytes, and the savings would be around 30%, which is still impressive. For large trees containing dictionaries or indexing large texts, we would be talking about hundreds of megabytes.

Figure 6.4 shows the best-case scenario for tries, but obviously things are not always looking this good. Figure 6.5 shows a different edge case, close to the worst-case scenario, where the longest prefix in a (degenerate) trie is the empty string. In cases like this, information

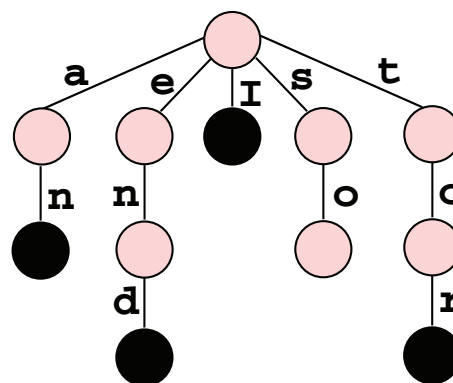


Figure 6.5 A degenerate trie, where no string shares a prefix with another string

ends up being stored very inefficiently; luckily, however, these edge cases are incredibly unlikely in real-world applications.

So much for space consumption. At worst, tries can be considered comparable to binary search trees. What about their running time? We will answer this question while looking at the individual methods in order to develop an understanding of how these results are derived.

6.2.2 Search

Let's start with search. Assuming we have built a proper trie, how do we check whether it contains a certain key?

Turns out, it's not too difficult, compared to a BST. The main difference is that we need to walk the tree one character (of the searched key) at a time, following the link that is marked precisely with that character.

Both strings and tries are recursive structures, whose unit of iteration is the single character; each string, in fact, can be described as either

- The empty string, ""
- The concatenation of a character c and a string s' : $s=c+s'$, where s' is a string one character shorter than s and can possibly be the empty string

NOTE In most programming languages single quote characters wouldn't be allowed in variable names, so we use `tail` in listing 6.2 as a substitute name for s' , which denotes the tail of current string s in the figures.

For instance, the string "home" is made of the character 'h' concatenated to the string "ome", which in turn is 'o' + "me", and so on, until we get to "e", which can be written as the character 'e' concatenated to the empty string "".

A trie, on the other end, stores strings as paths from the root to key nodes. We can describe a trie T as a root node connected to (at most) $|\Sigma|$ shorter tries. If a sub-trie T' is connected to the root by an edge marked with character c ($c \in \Sigma$), then for all strings s in T' , $c + s$ belongs to T .

For instance, in figure 6.3, the root has only one outgoing edge, marked with 'a'; considering T' as the only sub-trie of the root, T' contains the word "n", and this means that T contains 'a' + "n" = "an".

Since both strings and tries are recursive, it's natural to define the search method recursively. We can consider just the case where we search the first character of a string $s=c+s'$ starting from the root R of the trie T . If c , the first character of s , matches an outgoing of R , then we can search s' in the (sub)trie T' . We assume the root of the sub-trie is not null (as we'll see shortly, it's a reasonable assumption).

If at any point s is the empty string, then we have traversed the whole path in the trie corresponding to s . We then need to check the current node to verify whether our string s is stored in the tree.

If, instead, at some point the current node doesn't have an outgoing edge matching current character c , then we are sure string s is not stored in the trie.

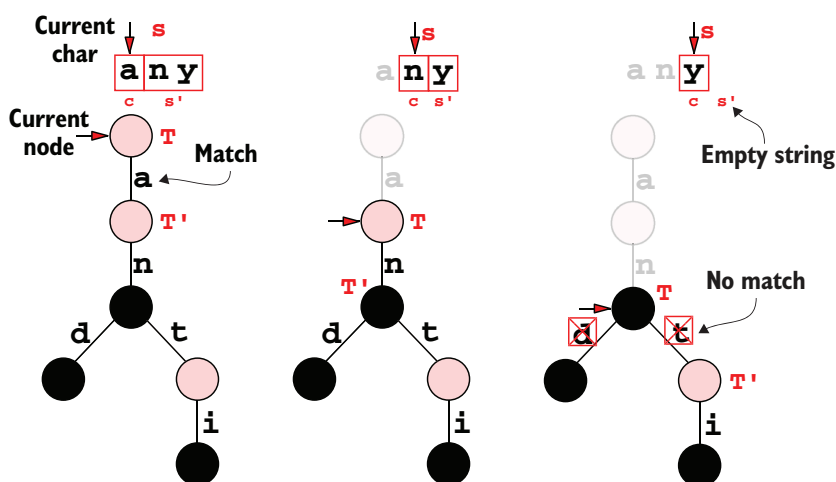


Figure 6.6 An unsuccessful search in a trie. At each step, we break the key to search, the string s , into $c+s'$: the concatenation of its first character and the rest of the string. Then we compare c to the current node's outgoing edges, and if we find a match, we go ahead traversing the tree. In this example, the search fails because the last character in the string doesn't match any outgoing edge.

We'll illustrate these examples in figures 6.6 and 6.7, but first, let's take a look at the implementation of the search method that we will then use as a reference while describing these examples.

In listing 6.2 we show a recursive implementation of the search method. At each call, we need to check if the first character in the substring searched matches an outgoing edge of the current node, and then recursively search the tail of the current string in the subtree referenced by that edge. Figures 6.6 and 6.7 shows the parallel between moving “right” in the string searched and traversing down the trie.

The same method can obviously be implemented using explicit loops. This implementation of the search method is likely suitable for a compiler's tail-call optimization,¹³ but as discussed in appendix E and chapter 3, if you are not comfortable with recursion or are not sure that your compiler will apply tail-call optimization, my advice is to write the iterative version of these methods to avoid stack overflow.

Listing 6.2 Method search

Method `search`, a standalone function, takes a trie node and the string key s to be searched. It returns `true` if s is stored in the trie, and `false` otherwise. We assume that `node` is never `null`, which is a reasonable assumption if this method is implemented as a private method internally called by the trie's API `search` method.

```
function search(node, s)
  if s == "" then
    return node.keyNode
```

Checks if the string searched is empty. If it is, since this method is implemented using recursion, we know it has traversed the whole path in the trie.

We are at the target node for the target key; the trie stores it only if the current node is a key node.

¹³As explained in appendix E, whenever in a method defined using recursion the recursive call is limited to the very last operation, compilers can optimize the target machine code by rewriting code using explicit loops instead of function calls. The caveat is that not all compilers provide this optimization.

Since *s* is not the empty string, it can be broken into a *head* character, *c* (the first character of *s*) and a *tail*, the rest of the string.

```

→ c, tail ← s.splitAt(0)
  if node.children[c] == null then
    return false
  else
    return search(node.children[c], tail)

```

If there is no outgoing edge in node for the character *c*, it means that we cannot traverse the trie any further, so that *s* is not stored in the subtree rooted at node.

Otherwise, we recursively search *tail* into the subtree referenced by *children[c]*.

Listing 6.3 shows the trie's API counterpart, which in turn will call the method in listing 6.2. We will omit these wrapper methods for the other operations when they are as trivial as for search.

Listing 6.3 Method `Trie::search`

Method `search` (for class `Trie`) takes a string key *s* to be searched; it returns `true` if *s* is stored in the trie, and `false` otherwise.

```

→ function Trie::search(s)
  if this.root == null then
    return false
  else
    return search(this.root, s)

```

Checks if the trie's root is `null`. If so, no string is stored in the trie, and we can return `false`.

Otherwise, we forward the call to the root's search method.

As already mentioned, there are two possible cases of unsuccessful search in tries: in the first one, shown in figure 6.6, we traverse the trie until we get to a node that doesn't have an outgoing edge for the next character. In the example, a call to `trie.search("any")`, this happens when we get to the key's last character, 'y' (as shown in the right diagram in figure 6.6). In listing 6.2, this corresponds to the condition in the `if` at line #5 returning `true`.

The other possible case for an unsuccessful search is that we always find a suitable outgoing edge until we recursively call `search` on the empty string. When this happens, it means we've reached the trie's node whose path from the root spells out the searched key. For instance, in figure 6.7 we traverse the tree link by link and check the string key character by character, until the condition at line #2 of listing 6.2 returns `true` and we go to line #3.

The result of line #3 is the only difference between a successful and an unsuccessful search. Looking at the example in figure 6.7, a successful search for the word "ant" would follow the exact same steps, with the only difference being that the final node (denoted with *T* in the rightmost diagram) would have to be a key node.

Notice that in listing 6.3, we can avoid checking for empty tries or handling the root as a special case, because in the trie constructor in listing 6.1, we create the root as an empty node. This (together with careful implementation of all the methods) will also support our assumption that the node argument in `search` (and all the other methods) is never `null`.

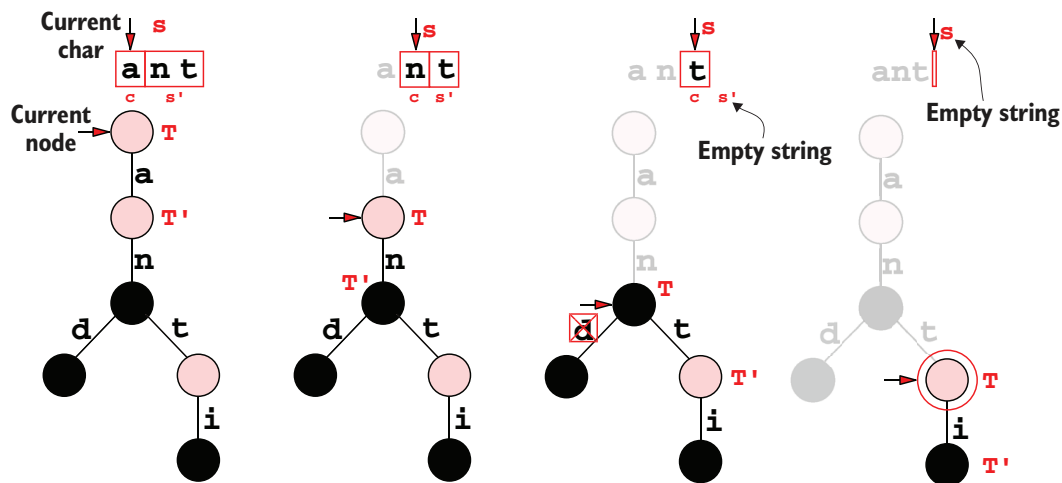


Figure 6.7 Another example of an unsuccessful search in a trie. In this example, the search fails because the path corresponding to the string searched ends in an *intermediate node*.

The search method is the most important method for tries, because all the other methods will be based on it. Search is so crucial to those implementations that we provide (in listing 6.4) a variant, `searchNode`, that returns the node found, rather than just true or false. We'll see in the next sections how this can be used to implement `remove`.

Listing 6.4 Method `searchNode`

```
function searchNode(node, s)
  if s == "" then
    return node
  c, tail ← s.splitAt(0)
  if node.children[c] == null then
    return null
  else
    return search(node.children[c], tail)
```

Performance-wise, how fast is `search`? The number of recursive calls we make is limited by the smaller of two values: the maximum height of the trie and the length of the search string. The latter is usually shorter than the former, but either way, for a string of length m we can be sure that no more than $O(m)$ calls are going to be made, regardless of the number of keys stored in the trie.

The key is, then, how long each step takes. It turns out there are three factors influencing this cost:

- The cost of comparing two characters: this can be assumed to be $O(1)$.
- The cost of finding the next node: for an alphabet Σ of size k , this can be, depending on the implementation:

- Constant (amortized or worst-case¹⁴), $O(1)$, using hash tables for edges
- Logarithmic worst-case $O(\log(k))$, using a balanced tree
- Linear worst-case $O(k)$, using plain arrays

Amortized constant time can reasonably be assumed in most cases.

- The cost of following a link and of splitting a string into `head+tail`. This point is the one where we need to be really careful. The naïve approach of extracting a substring for each node would be a performance disaster in most programming languages. Strings are usually implemented as immutable objects, so extracting a substring would require linear time and extra space, $O(m)$, for each call. Luckily, the solution is simple: we can pass to the recursive call a reference to the beginning of the string and the index of the next character. This way, even this operation can be considered $O(1)$.

Since each call can be implemented in such a way as to require amortized constant time, the whole search takes $O(m)$ amortized running time.

6.2.3 Insert

Like search, `insert` can be better defined recursively. We can identify two different scenarios for this method:

- The trie already has a path corresponding to the key to be inserted. If that's the case, we only have to change the last node in the path, making it a key node (or, if we are indexing a text, adding a new entry to the list of indices for the word).
- The trie only has a path for a substring of the key. In this case, we will have to add new nodes to the trie.

Figure 6.8 illustrates an example of the former situation, where a call to `insert("anthem")` on the trie in figure 6.7 will result in adding a new branch to one of the leaves of the tree.

The method, as shown in listing 6.5, mainly consists of two steps. In the first step, we traverse the tree, following links corresponding to the next character in the key to add, until we either have consumed the whole input string, or we get to a node where there is no outgoing edge matching the key's next character.

Using listing 6.5 as a reference, we can see that the first step of the algorithm is implemented in lines #1 to #7, where we keep traversing the tree using the characters in the key to insert to choose the next branch to traverse. This is exactly the same as for search, with one difference: if we consume all characters in the input string (meaning that we traversed the whole path from the root, and reached the target trie node) we just have to set the node at the end of the path to a key node. This corresponds to the first scenario described at the beginning of the section (not shown in figure 6.8).

¹⁴Since the set of keys for the hash table, that is, the alphabet, is static and known in advance, it is possible to use perfect hashing and obtain worst-case constant time lookup. See appendix C for more details.

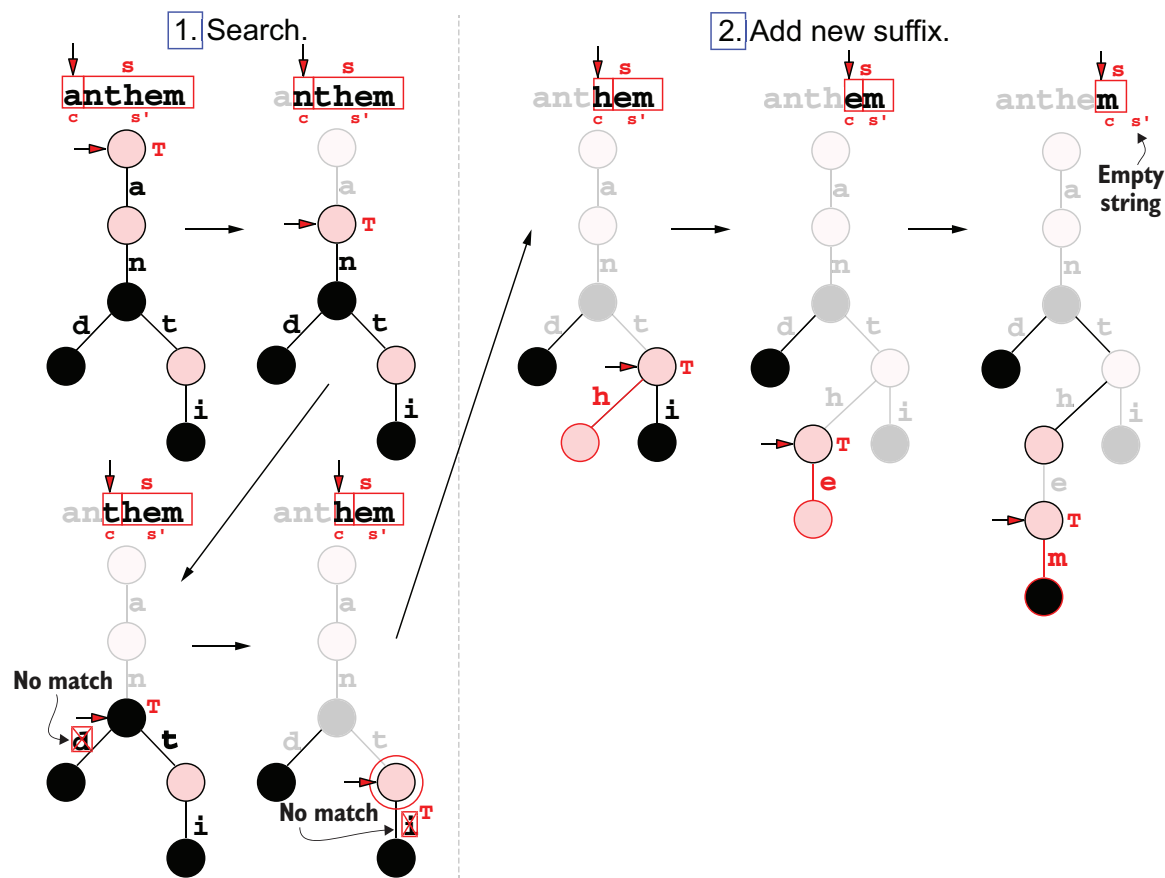


Figure 6.8 An example of method `insert`. In a call to `trie.insert("anthem")`, we first search the longest prefix of `s` in the trie ("ant"); then, as a second step, from the node corresponding to the longest common prefix we add a new path for the remainder of `s` ("hem").

When we reach the status shown in the last diagram of the left half of figure 6.8, it means that the condition at line #6 of listing 6.5 has become `false`. If that's the case, we need to jump to line #9 and add a brand new branch to the tree for the remaining characters in the string: by doing so, we are adding a suffix to the string matching the path from root to current node (in the example, we will add the suffix "hem" to the string "ant" already in the trie).

Listing 6.5 Method `insert`

Method `insert` takes a trie node and the string key `s` to be inserted. It returns nothing but has side effects on the trie. Again, we assume that `node` is never `null`: this is a reasonable assumption if this method is implemented as a private method, internally called by the trie's API `insert` method.

```
function insert(node, s)
  if s == "" then
    node.keyNode ← true
  return
```

Checks if the string searched is empty. If it is, since this method is implemented using recursion, we know it has traversed the whole path in the trie.

We are at the target node for the target key. We set current node to a key node to ensure it will store `s`.

Since s is not the empty string, it can be broken into a *head* character, c (the first character of s) and a *tail*, the rest of the string.

```

→ c, tail ← s.splitAt(0)
  if node.children[c] != null then
    return insert(node.children[c], tail)
  else
    return addNewBranch(node, s)

```

If there is an outgoing edge in *node* for the character c , we can keep recursively traversing the tree.

Following the recursive definition of this method, we need to insert *tail* in the subtree referenced by the edge marked with character c .

Otherwise, it means that we cannot traverse the trie any further: now we have to add the remaining characters in s as a new branch. (Be careful: not just the ones in *tail*! We also need to include character c .)

This last operation is implemented in a different utility method that is shown in listing 6.6 using (surprise!) recursion.

The definition of the method is quite straightforward. As shown in the right half of figure 6.8, we just consume one character of the remaining string, create a new edge marked with this character and a brand new, empty node N at the other side of the edge, and then recursively add the tail of the string to the tree T rooted at N .¹⁵

Listing 6.6 Method `addNewBranch`

Method `addNewBranch` takes a trie node and the string key s for the new branch. As always, we assume that *node* is never `null`.

```

→ function addNewBranch(node, s)
  if s == "" then
    node.keyNode ← true
    return

```

Checks if the string to add is empty. If it is, we've added all the edges needed to the new branch, and current node is the last node in the path.

Therefore, to complete the insertion, we just need to set current node to a key node.

```

  c, tail ← s.splitAt(0)
  node.children[c] ← new Node(false)
  return addNewBranch(node.children[c], tail)

```

Since s is not the empty string, it can be broken into a *head* character, c (key first character of s) and a *tail*, the rest of the string.

We add a new outgoing edge to current node, marked with character c . At the other end of the edge, we create a new empty node.

We recursively add the characters in *tail* as a new branch of `children[c]`.

Similarly to what we did for `search`, we can prove that `insert` also takes $O(m)$ amortized time, if the creation of a new node can be performed in constant time (which is the case if we use hash tables for edges, but not if we use plain arrays).

6.2.4 Remove

When it comes to removing a key from a trie, we are in the position to choose. We can go for an easier, cheaper algorithm that will cause the tree to grow beyond what's necessary, or we can implement the full method, more complicated and possibly slower in practice, but with the lowest impact on memory.

¹⁵Notice that, since we just created this node N , it will be the only node in its subtree.

The difference between the two alternatives is that the first one simply unmarks a key node, making it an intermediate node, and doesn't worry about the tree structure. This can be easily implemented reusing the `searchNode` method in listing 6.4, shown in listing 6.7.

Listing 6.7 Method `Trie::remove` (no pruning)

Method `remove` (for class `Trie`) can be implemented naively for the `Trie` class using `search`. Here it takes the key to remove and returns a `Boolean` conveying the information about whether the key has been found and deleted.

```
function Trie::remove(s)
  node ← searchNode(this.root, s)
  if node == null or node.keyNode == false then
    return false
  else
    node.keyNode ← false
    return true
```

Performs a search on the trie and gets the node for `s` (if present)

If node is null, or it's not a key node, it means that the key was not stored in the trie, so we return false.

Otherwise, we mark the node as an intermediate node, and return true.

What's the issue with this method? Take a look at figure 6.8. If we just transform a key node `N` into an intermediate node, we can have two possible situations:

- `N` is an inner node, so it will have children storing keys that can only be reached by passing through `N` itself.
- `N` is a leaf, which means that there is no key stored in the trie that can only be reached through `N`.

The case where `N` is a leaf is illustrated in figure 6.9. After “unmarking” the key node at the end of the path, we can see that the trie has a “dangling” branch that contains no key. While it is perfectly fine to leave such a branch, because all the methods for manipulating tries will still work, if data is dynamic and there is a large ratio of deletions on the trie, the amount of memory wasted in dangling branches can become significant.

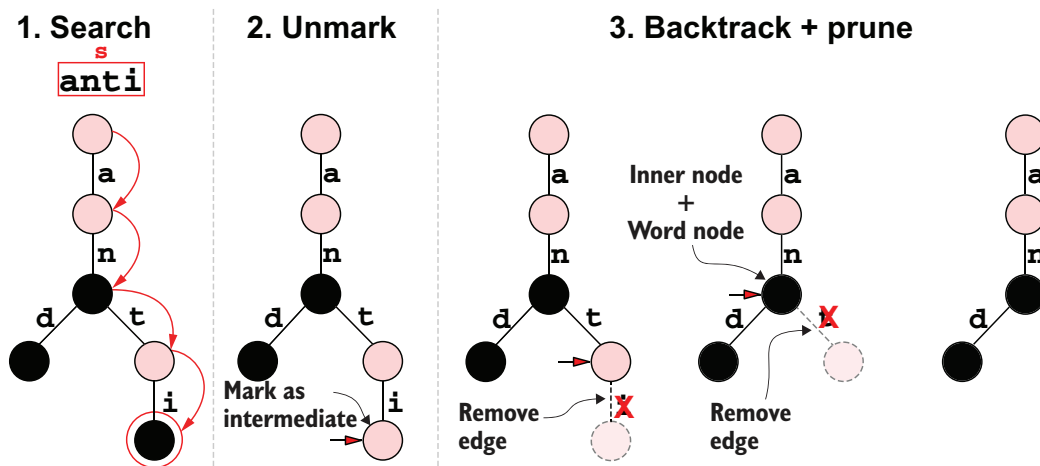


Figure 6.9 An example of method delete. (1) Find the key to delete. (2) Mark the node at the end of the path as “intermediate node.” (3) Prune the tree to remove dangling branches.

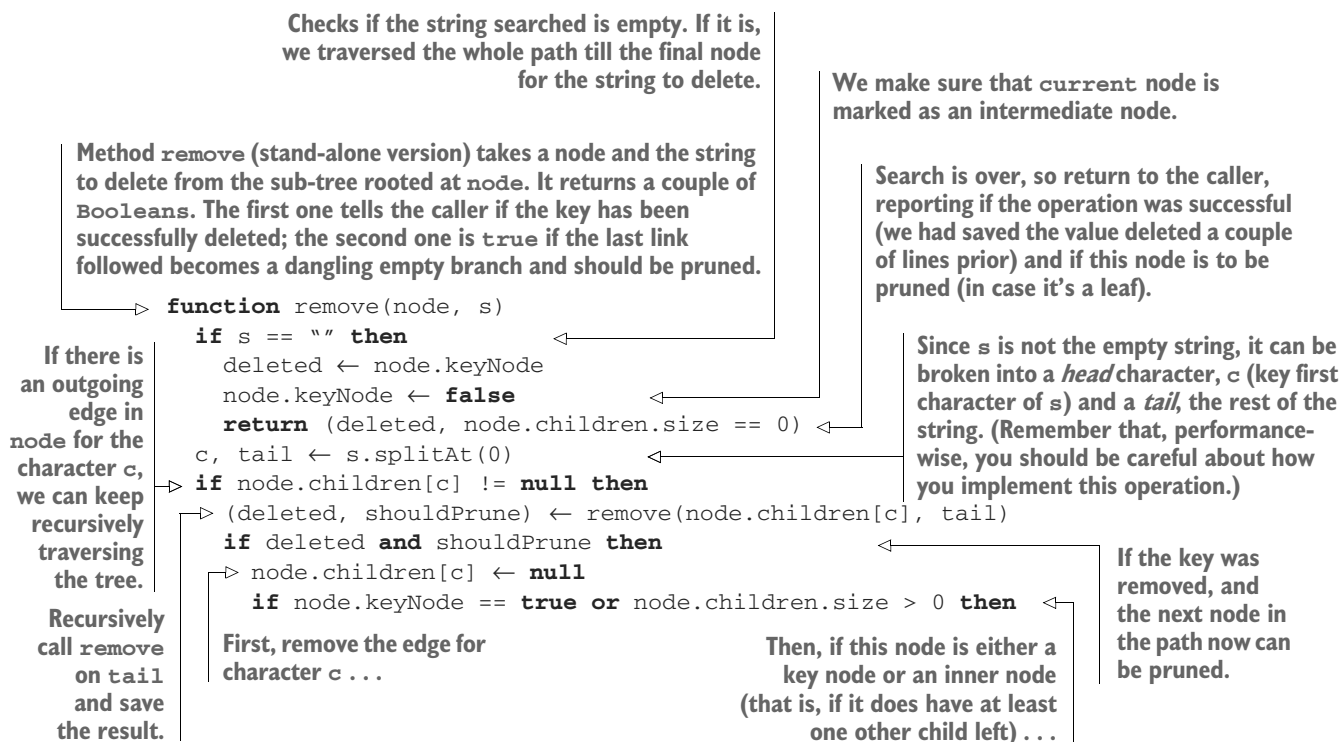
The solution, in these cases, is implementing a pruning method that will remove dangling nodes while backtracking the path traversed during search. If we have the guarantee that the trie is “clean,” meaning there were no dangling branches before removing the current node, there are just two conditions for which we would stop pruning while backtracking:

- We reach a key node. Obviously, we can’t remove a node holding a key.
- We reach an inner intermediate node. After removing the last edge in the path being backtracked, if the current node becomes a leaf, it can be removed; otherwise, if this node has other children, then all its sub-branches will hold at least a key (because of our premise) and therefore the current node corresponds to an intermediate character in one or more strings stored in the trie, and it can’t be deleted.

Listing 6.8 shows the implementation of this method performing deletion + pruning. Using figure 6.9 as a reference, you can see that, after turning it to an intermediate node (line #4), the node at the end of the path “anti” becomes a worthless leaf. We backtrack to its parent and remove the edge marked with ‘i’ (lines #9 to #12 in listing 6.8), and then the node at the end of the path for “ant”, which also is an intermediate node, becomes a leaf too, and can thus be removed.

When we backtrack once more, we can see that its parent is both a key node, and has another child, so we can’t prune the tree anymore.

Listing 6.8 Method `remove` (with pruning)



```

... it means that this node can't be pruned
anymore, so we update the flag before returning.
    shouldPrune ← false
    return (deleted, shouldPrune)
else
    return (false, false)

```

If execution gets here, it means we haven't found the key in the trie, so it can't be deleted.

This version of removal obviously needs a different implementation of the trie method than the naïve one shown in listing 6.7. In this case, though, the API method is even simpler than the one shown in listing 6.7; basically, it just becomes a wrapper.

Performance-wise, the same considerations made for search and insert apply for remove. If we implement the method with pruning, the number of operations on the tree is at worst $2 \cdot m$, twice as much as for the naïve version without pruning, for which at most m edges are traversed. Execution time is also probably going to be more than two-fold, because, as you can see from the code, the delta in code complexity is relevant.

The tradeoff for faster running time, however, is that the tree can grow significantly if we don't prune dangling branches; the best choice depends on your requirements and context. If you have a dynamic set and expect many calls to delete, you'd better use pruning. If, instead, the ratio of insertion/removal is low, or you expect to have to add back deleted strings frequently or shortly after, then you are better off with the faster (though messier) removal shown in listing 6.7.

6.2.5 Longest prefix

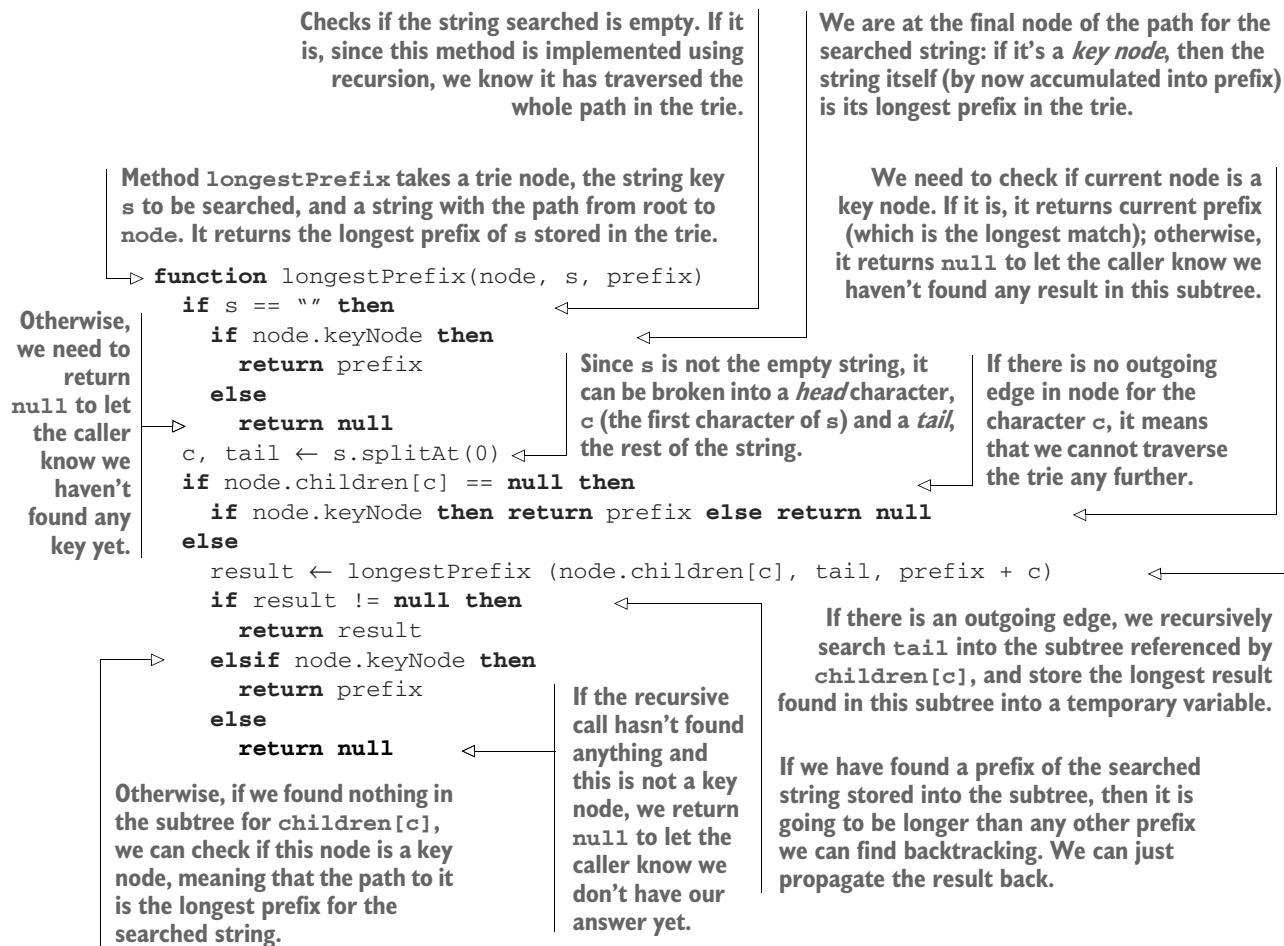
With remove, we completed our overview of the classic operations on containers. As we mentioned, though, tries can also provide two new kind of operations, which are the most exciting parts of this data structure.

In this section we'll focus on the method that returns the longest prefix of the searched string; given an input string s , we traverse the trie following the path corresponding to characters in s (as long as we can), and return the longest key we found.

Sometimes, even if a key wasn't stored in our trie, we are just interested in getting its longest prefix. We'll see an example in the applications section.

The search for the longest prefix is almost entirely the same as the search method. The only difference is that in a recursive implementation, when we backtrack we need to check whether we have already found a key, and if we haven't and the current node is a key node, we'll have to return the current node's key. This also means that we have to return a string, not just true or false, and at each call keep track of the path traversed, because we need to know what we can return. Since backtracking walks the path backward, the first key node we find while backtracking will hold the longest prefix.

Listing 6.9 clarifies these concepts by showing the implementation of this method. If you recall how insert works, it can be rethought as a two-step operation: finding the longest common prefix of the key to be inserted already in the trie, and then adding a branch with the remaining characters. As an exercise, try to rewrite the pseudo-code for insert by leveraging method longestPrefix.

Listing 6.9 Method `longestPrefix`

As with the other methods we have described so far, this operation is also linear in the length of the searched string: $O(m)$, if $|s| = m$.

6.2.6 Keys matching a prefix

The last method we are going to describe returns all the keys matching a certain prefix.

If you stop and think about the definition of a trie, the implementation for this method will flow almost naturally. Even the alias for this data structure, *prefix tree*, suggests a solution. We have seen, in fact, that tries compactly store strings sharing the same prefix, because each string is translated in a path from the root to a key node, and strings sharing the same prefix will result in sharing the same path in the trie.

For instance, in figure 6.3, all the strings "and", "ant", "anthem" share a portion of their path, corresponding to the common prefix "an".

Listing 6.10 shows the implementation of this method. Not surprisingly, it's one more method that leverages the `searchNode` method defined in listing 6.4.

Listing 6.10 Method `Trie::keysStartingWith`

Method `keysStartingWith` for the `Trie` class takes a string `prefix`, and returns the list of all keys stored in the trie that starts with this prefix.

```
function Trie::keysStartingWith(prefix)
  node ← searchNode(this.root, prefix)
  if node == null then
    return []
  else
    return allKeys(node, prefix)
```

Otherwise, we have to return all keys stored in the sub-tree rooted at `node`.

Performs a search on the trie and gets the node for `prefix` (if present). Remember that `searchNode` will return the node at the end of the path, even if it's an intermediate node, or `null` if there isn't any such path.

If `node` is `null`, it means that there is no key stored in the trie that starts with `prefix`, so we return an empty list.

Clearly, there is a new method that we still need to define: `allKeys`, the method that traverses a (sub)trie and collect all its keys; this method, shown in listing 6.11, is a traversal of the whole subtree. We traverse all branches for each node, and we only stop following a path when we reach a leaf. We also need to pass the (string corresponding to the) path traversed so far, up to `node`, as the second argument, because we will need that to know which key we should return.

Listing 6.11 Method `allKeys`

Method `allKeys` takes a trie node and the string `prefix` corresponding to the path from the trie's root to `node`. It returns the list of strings $s_k = \text{prefix} + \text{suffix}_k$, where suffix_k is the k -th string contained in this subtree.

```
function allKeys(node, prefix)
  keys ← []
  if node.keyNode then
    keys.insert(prefix)
  for c in node.children.keys() do
    keys ← keys + allKeys(node.children[c], prefix + c)
  return keys
```

Returns all keys gathered

If current node is a key node, then we need to add `prefix` to the list of strings contained in the subtree rooted at `node`—assuming `prefix` is the correct string for the path from root to current node.

Iterate over `node`'s outgoing edges; in particular, we need the characters marking each edge.

Add to the list of keys for the subtree rooted at `node` all the keys in the subtree referenced by the edge. For this subtree, the path from the root will be made by `prefix + c`. Be careful about the implementation of this operation; it can be costly if not implemented properly.

When we run the asymptotic analysis for this method, we need to be especially careful with line #6. Depending on the programming language and data type used, concatenating lists can be quite expensive, if it's not done right.

The most efficient way to accumulate the keys found would be to pass a third parameter to the method, an accumulator, to which we would add each key only once and in one place, line #4.

Under this assumption, the running time for method `allKeys` is $O(j)$, for a trie with j nodes, and therefore the worst-case upper bound for the method `keysStartingWith` is $O(m+j)$, for a trie with j nodes and a prefix with m characters.

The caveat is that it's hard to know or even estimate how many nodes a trie will have based on the number of keys it stores. If, however, we know it contains n keys whose maximum length is M , then the worst-case (loose) bound for a non-empty string is $O(m + n * (M - m))$, corresponding to a degenerate trie where all words share exactly the prefix searched, and no further character.

In the example shown in figure 6.10, we search for all keys matching prefix "ant", so $n=6$, $m=3$ and $M=8$ (the length of the longest key, "antidote").

If, instead, we search all keys starting whose prefixes include the empty string, this will return all keys in the tree, and the running time will be $O(n * M)$, which would also be the worst-case upper bound for the method.

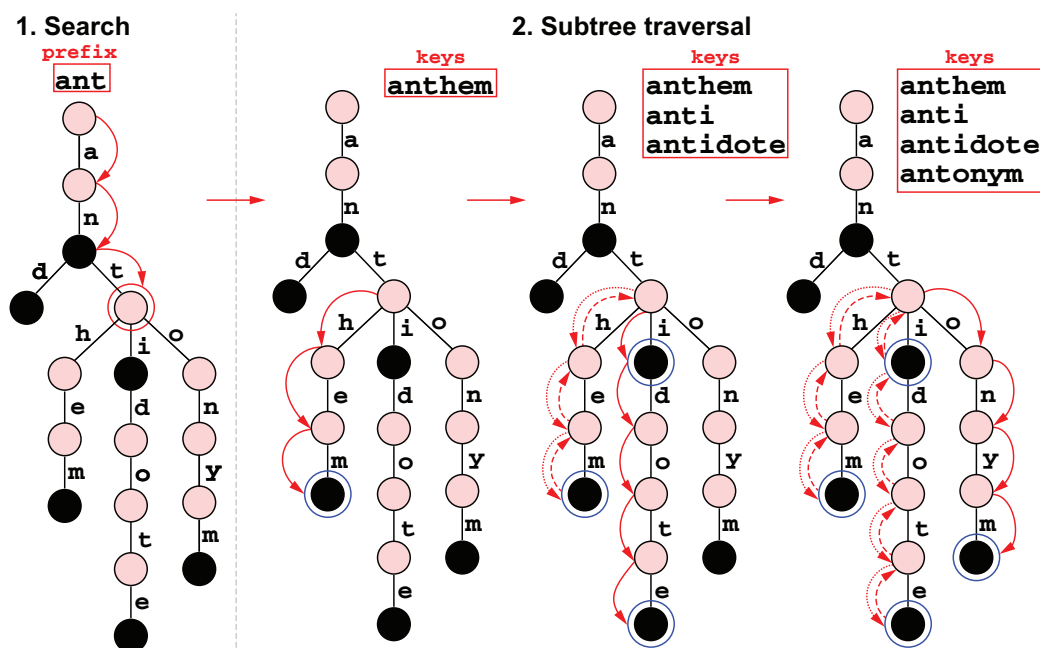


Figure 6.10 An example of method `keysWithPrefix`. (1) Traverse the path corresponding to the common prefix. (2) Collect all keys found traversing the subtree at the end of the path for prefix.

6.2.7 When should we use tries?

Now that we have described all the main methods on tries, it feels like taking a moment to recap would be a good idea. Table 6.2 shows the performance of tries on these methods, compared to the equivalent methods for balanced BSTs.

Table 6.2 helps answer the question about performance that we put on hold in section 6.2.1. We saw when a trie would require less memory than a BST; now we also know that it would almost always be faster.

Remember that while in general we express the running time for BSTs in terms of n , the number of entries stored, in this case we can't assume that the cost to compare two keys is $O(1)$, but it's rather $O(m)$, depending on the length m of the shortest of the two keys.

Table 6.2 Running time for operations on tries vs balanced BSTs, assuming n keys with average length m ; finally, as a simplification we assume for the size of the input keys m , that $m \in O(M)$

Method	BST	BST + hash	Trie
Search	$O(m \cdot \log(n))$	$O(m + \log(n))$	$O(m)$
insert	$O(m \cdot \log(n))$	$O(m + \log(n))$	$O(m)$
remove	$O(m \cdot \log(n))$	$O(m + \log(n))$	$O(m)$
longestPrefix	$O(m \cdot n)$	$O(m + n)$	$O(m)$
keysWithPrefix	$O(m \cdot n)$	$O(m + n)$	$O(n + m)^a$

^aaverage

The third column in table 6.2 shows the results for a particular variant of BSTs where we store a hash of the string, together with the key itself, in each node. This approach, which requires an extra $O(n)$ memory to store these fields, allows for a fast two-pass comparison. Given a search string w , we compute $h(w)$ before starting the search. Then for each node we first check whether $h(w)$ matches the node's hash (which requires constant time), and only when it does do we perform a proper strings comparison.

Before moving on, let's also recap the pros and cons of using tries, and when we should prefer a trie over a BST.

On the pros side, compared to using BSTs or hash tables

- The search time only depends on the length of the searched string.
- Search misses only involve examining a few characters (in particular, just the longest common prefix between the search string and the corpus stored in the tree).
- There are no collisions of unique keys in a trie.
- There is no need to provide a hash function or to change hash functions as more keys are added to a trie.
- A trie can provide an alphabetical ordering of the entries by key.

As appalling as this list looks, as we have repeated many times, unfortunately there is no perfect data structure. So even tries do have some downsides:

- Tries can be slower than hash tables at looking up data whenever a container is too big to fit in memory. Hash tables would need fewer disk accesses, even down to a single access, while a trie would require $O(m)$ disk reads for a string of length m .
- Hash tables are usually allocated in a single big and contiguous chunk of memory, while trie nodes can span the whole heap. So, the former would better exploit the principle of locality.
- A trie's ideal use case is storing text strings. We could, in theory, stringify any value, from numbers to objects, and store it. Yet, if we were to store floating

point numbers, for instance, there are some edge cases that can produce long meaningless paths,¹⁶ such as periodic or transcendent numbers, or results of certain floating points operations such as $0.1+0.2$, due to issues with double precision representation.¹⁷

- Tries have memory overhead for nodes and references. As we have seen, some implementations require each node to store an array of $|\Sigma|$ edges, where Σ is the alphabet used—even if the node has few or no children at all.

In summary, the advice could be to use tries when you have to frequently perform prefix searches (`longestPrefix` or `keysWithPrefix`). Use hash tables when data is stored on slow supports like disk or whenever memory locality is important. In all intermediate cases, profiling can help you make the best decision.

Tries offer extremely good performance for many string-based operations. Due to their structure, though, they are meant to store an array of children for each node. This can quickly become expensive. The total number of edges for a trie with n elements can swing anywhere between $|\Sigma| * n$ and $|\Sigma| * n * m$, where m is the average word length, depending on the degree of overlap of common prefixes.

We have seen that we can use associative arrays, dictionaries in particular, to implement nodes, only storing edges that are not `null`. Of course, this solution comes at a cost: not only the cost to access each edge (that can be the cost of hashing the character plus the cost of resolving key conflicts), but also the cost of resizing the dictionary when new edges are added.

6.3 Radix tries

To overcome these issues with tries, a few alternatives have been developed: the *ternary search trie* (TST), which trades off lower memory usage for worse running time, or the *radix trie*, just to name a few.

While TSTs improve the space requirements to store links, and free us from worrying about platform-specific implementations to optimize how we store edges, the number of nodes we need to create is still on the order of magnitude of the number of characters contained in the whole corpus stored, $O(n * m)$ for n words of average length m .

In tries, most of the nodes don't store keys and are just hops on a path between a key and the ones that extend it. Most of these hops are necessary, but when we store long words, they tend to produce long chains of internal nodes, each with just one child. As we saw in section 6.2.1, this is the main reason tries need too much space, sometimes more than BSTs.

Figure 6.11 shows an example of a trie. Nothing special, just a small, regular trie. We can see that intermediate nodes always have children (assuming we prune dangling branches after deleting keys); sometimes just one child, sometimes more.

¹⁶See <http://stackoverflow.com/questions/588004/is-floating-point-math-broken/27030789#27030789>.

¹⁷See https://en.wikipedia.org/wiki/IEEE_floating_point#Basic_formats.

When an intermediate node has more than one child, we have several branches that we can traverse from it. When, instead, there is just one child, those two nodes begin to resemble a linked list. For example, take the first three nodes from the root of figure 6.11: they encode the prefix “an”, and the search of any other string starting with ‘a’ but not followed by an ‘n’ couldn’t get anywhere in the tree.

In fact, it turns out that an intermediate node is a branching point if it has more than one child: it means that the trie stores at least two keys sharing the common prefix corresponding to that node. If that’s the case, the node carries valuable information that we can’t compress in any way.

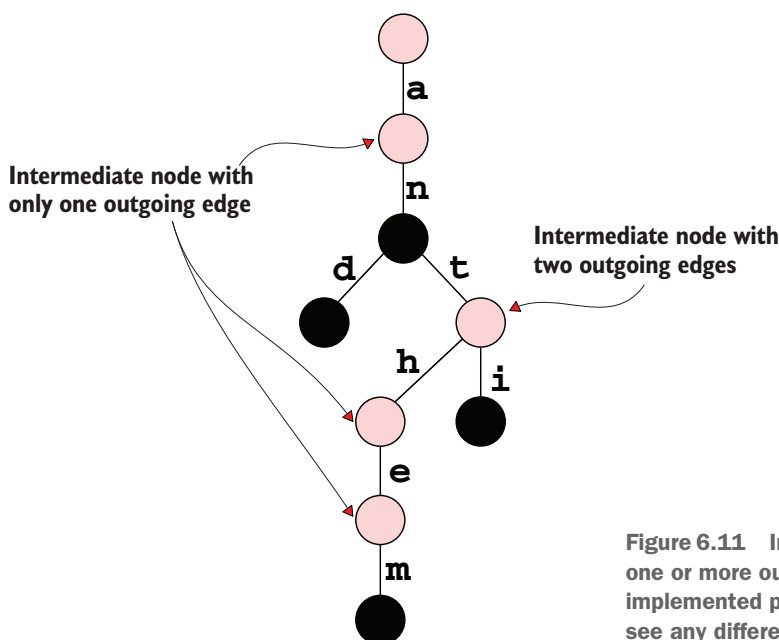


Figure 6.11 Intermediate nodes in tries have one or more outgoing edges (assuming we implemented pruning after delete). Do you see any difference?

Key nodes also store information, regardless of the number of children they have. They tell us that the path to reach them composes a string that is stored in the trie.

If, however, an intermediate node stores no key and only has one child, then it carries no relevant information; it’s only a forced step in the path.

Radix tries (aka *radix trees*, aka *Patricia trees*¹⁸) are based on the idea that we can somehow compress the path that leads to this kind of nodes, that are called *pass-through nodes*.

How? Figure 6.12 gives a hint about the process to compress these paths. Every time a path has a pass-through node, we can squash the section of the path hinging on these nodes into a single edge, which will be labeled with the string made concatenating the labels of the original edges.

¹⁸The original name for this DS, Patricia tree, is an acronym. Morrison, Donald R. “PATRICIA—practical algorithm to retrieve information coded in alphanumeric.” *Journal of the ACM (JACM)* 15.4 (1968): 514-534.

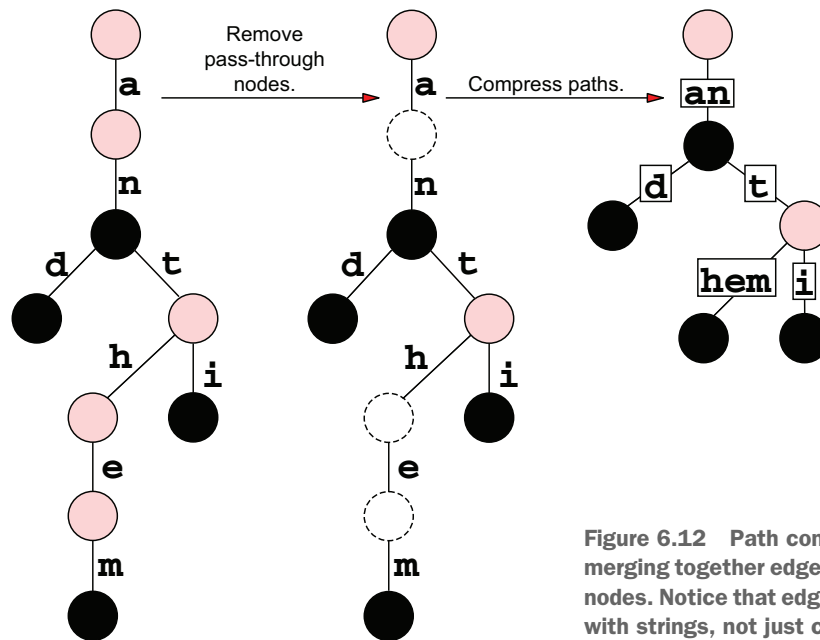


Figure 6.12 Path compression in tries, by merging together edges adjacent to pass-through nodes. Notice that edges in radix tries are labeled with strings, not just characters.

How much can we save with this change? Let's look at the two trees in figure 6.12 to get an idea.

The original trie has 9 nodes and 8 edges, and with the assumptions made in section 6.2.1, with a 4-byte overhead per node, this means $9 * 4 + 8 * 9 = 108$ bytes.

The compressed trie on the right has 6 nodes and 5 edges, but in this case each edge carries a string, not just a character; however, we can simplify the operation by accounting for edge references and string labels separately. This way, we would still count 9 bytes per edge (because we would include the string terminator byte in the edge cost), but we could add the sum of string lengths as a third term in the final expression; the total number of bytes needed is given by $6 * 4 + 5 * 9 + 8 * 1 = 77$ bytes.

In other words, for this simple trie, the compressed version requires 30% less memory.

6.3.1 Nodes and edges

All the operations that we have described for tries can be similarly implemented for radix trees, but instead of edges labeled by chars, we need to store and follow edges labeled by strings.

While at a high level the logic of the methods is almost the same as for trie, to check which branch we should traverse, we can't just check the next character in the key, because edges could be labeled with a substring that matches more than one character of our argument *s*.

One important property in these trees is that no two outgoing edges of the same node share a common prefix. This is crucial and allows us to store and check edges more efficiently.

A first solution is keeping edges in sorted order and using binary search to look for a link that starts with the next character c in the key. Because there can't be two edges starting with c , if we find one, we can compare the rest of the characters in its label to the next characters in the string. Moreover, binary search allows us to find this edge in logarithmic time in the number of edges, and because there can't be more than $k = |\Sigma|$ edges per node (because there can be at most one starting with each character in our alphabet Σ), we know that the worst case running time for performing binary search and finding a candidate edge is $O(\log(k))$.

Because k is a constant that doesn't depend either on the number of keys stored in the trie or on the length of the words searched/inserted/etc., we can consider $O(\log(k)) = O(1)$ as far as asymptotic analysis is concerned. Moreover, no extra space¹⁹ is required to store edges with this solution.

This solution is illustrated in figure 6.13, where we also show how binary search works to find the possible edge matching.

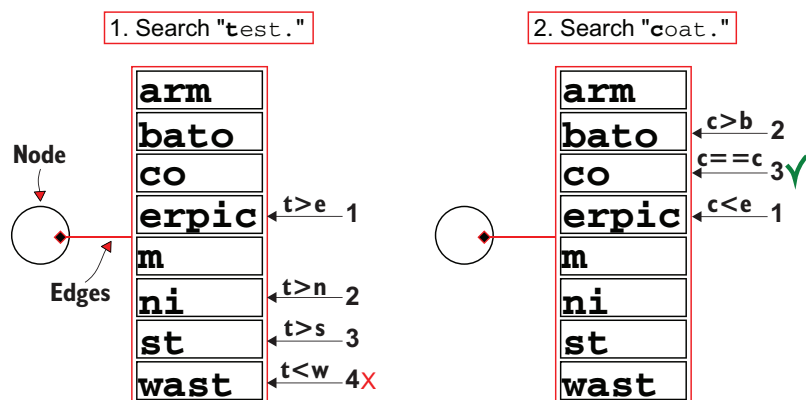


Figure 6.13 An example of a radix trie node with edges stored as in sorted array. (Left) A failed binary search. (Right) A successful search. Comparisons are performed only on the first character of each string: in the successful search, though, the edge's label is a prefix of the searched string.

Notice that the match between the string searched and an edge's label doesn't have (and usually isn't) full; we'll see in a moment what this means for our algorithms and how to handle these situations.

Of course, using sorted arrays, as we discussed in chapter 4 and appendix C, means logarithmic search, but linear (translated: slow!) insertion. Although the number of elements can be considered a constant, in asymptotic analysis, from a practical point of view this implementation can significantly slow down insertion of new keys in large tries.

The alternative solutions to implement this dictionary for edges are the usual: balanced search trees, which would guarantee logarithm search *and* insertion, or hash

¹⁹Except a constant overhead for the array object, in most languages.

tables. The latter is illustrated in figure 6.14. We can keep a dictionary whose keys are characters and whose values are the full string labels of the node's edges, together with a reference to the children linked by the edge. This solution requires $O(k)$ additional space for a node with k children, and worst-case it will require $O(|\Sigma|)$ extra space per node.

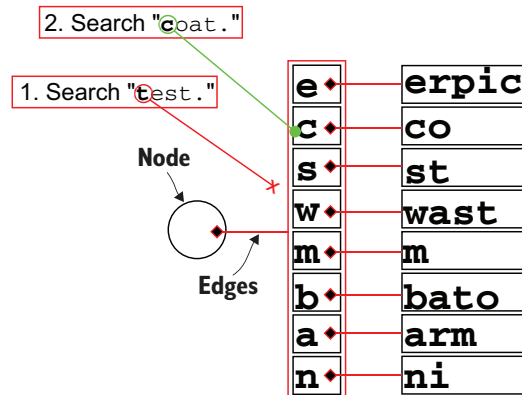


Figure 6.14 An example of a node where edges to children are stored in a dictionary. Dictionary's keys are characters, the first letter of each label, while values contain the full labels and the references to the destination node of the edge. The figure shows a simplification of the same searches in figure 6.13. As before, comparisons are based on the first character of the searched strings.

Despite requiring more space and a little bookkeeping on insertion and deletion to update the hash table, this solution allows amortized constant-time lookup when searching the path for a key.

Independently of the implementation, the first step will be comparing the first character of the input string to the first character of the edges' label.

Overall, we have four possible cases, illustrated in figure 6.15:

- 1 The string s_E labeling an edge perfectly matches a substring of s , the input the string. This means s starts with s_E , so we can break it up as $s = s_E + s'$. In this case, we can traverse the edge to the children, and recurse on the input string s' .
- 2 There is an edge starting with the first character in s , but s_E is not a prefix of s ; however, they must have a common prefix s_p (at least one character long). The action here depends on the operation we are running. For search, it's a failure because it means there isn't a path to the searched key. For insert, as we'll see, it means we have to decompress that edge, breaking down s_E .
- 3 The input string s is a prefix of the edge's label s_E . This is a special case of point #2 and can be handled similarly.
- 4 Finally, if we don't find a match for the first character, then we are sure we can't traverse the trie any longer.

Now that we have clarified the high-level structure of radix trie's nodes, let's delve into the algorithms. Keeping in mind the considerations we just discussed, their behavior will flow naturally from trie's methods.

Listing 6.12 shows the pseudo-code for the `RadixTrie` and `RTNode` classes, used to model this new data structure. We also added a class to model edges, to make code

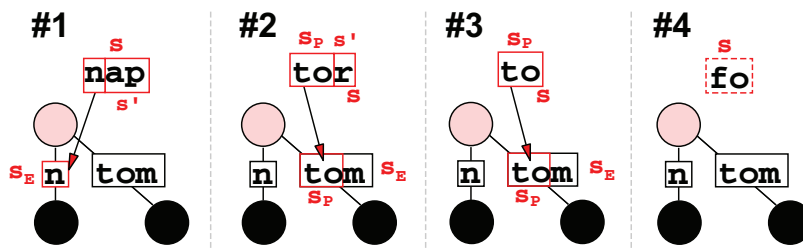


Figure 6.15 Possible cases comparing a search string to a node's links. (1) An edge's label completely matches part of the string. (2) An edge's label and the search string have a common prefix that is shorter than both strings. (3) The search string is a prefix of one of the edge's labels. (4) The search string has no common prefix with any of the edges.

cleaner. I wonder if you have noticed a tiny, but meaningful, detail: we don't need to define a fixed alphabet beforehand, like for tries!

You can also take a look at a full implementation on the book's repo on GitHub.²⁰

Listing 6.12 Class RadixTrie

```
class RTEdge
  #type RTNode
  destination

  #type string
  label

class RTNode
  #type boolean
  keyNode

  #type HashMap<Char, RTEdge>
  children

  function RTNode(storesKey)
    children ← new HashMap()
    this.keyNode ← storesKey

class RadixTrie
  #type RTNode
  root

  function RadixTrie()
    root ← new RTNode(false)
```

6.3.2 Search

The search method, shown in listing 6.13, is almost identical to the trie's counterpart; the only difference is the way we get the next edge to traverse. Because we are going to

²⁰See <https://github.com/mlarocca/AlgorithmsAndDataStructuresInAction#radix-trie-aka-patricia-tree>.

reuse this operation over and over again for the other methods, we extract its logic into a utility method, shown in listing 6.14.

Listing 6.13 Method search for radix tries

Method `search` takes an `RTNode` and the string key `s` to be searched. It returns `true` if `s` is stored in the trie, or `false` otherwise. We assume that `node` is never `null`: this is a reasonable assumption if this method is implemented as a private method, internally called by the `RadixTrie`'s API `search` method.

Checks if the string searched is empty. If it is, since this method is implemented using recursion, we know it has traversed the whole path in the tree and we reached the target node.

We are at the target node for the target key: the tree stores it only if current node is a key node.

```

function search(node, s)
  if s == "" then
    return node.keyNode
  else
    (edge, commonPrefix, sSuffix, edgeSuffix) ← matchEdge(node, s)
    if edge != null and edgeSuffix == "" then
      return search(node.children[commonPrefix].destination, sSuffix)
    else
      return false
  
```

Since `s` is not the empty string, we can check if there is an edge matching it, even partially.

Otherwise, we are in one of cases #2–4. The key is certainly not stored in the tree, and we can return.

If there is an edge sharing a common prefix with `s`, and the whole edge's label is a prefix to `s`, then we recursively search the remaining characters of `s` (stored in `sSuffix`) into the subtree linked by the edge. This is case 1 of the four possible matches in figure 6.15.

This method just looks for an edge with a common prefix with the target string `s`, if any. Remember that all edges can't have any prefix in common, so there can be at most one starting with the same character as `s`.

It returns some useful information that the caller can use to decide the action to take: the longest common prefix between searched string and edge's label, and the suffixes of these two strings (with respect to their common prefix).

Listing 6.14 Method `matchEdge` for Radix Tries

Method `matchEdge` takes an `RTNode` and the string key `s` to be matched. It returns a tuple with the edge matched, if any, the common prefix between `s` and the edge's label, and the suffixes of those strings. We assume that `node` is never `null` and that `s` is not empty.

Since `s` is not the empty string, it will certainly have a first character `c`.

Looks up in the hash table for `children` if there is any edge whose label starts with `c`

If there isn't any, it means there is no edge with a common prefix to `s`. Then, it returns `null` for the edge and an empty string for the common prefix, and consequently computes the suffixes.

```

function matchEdge(node, s)
  c ← s[0]
  if node.children[c] == null then
    return (null, "", s, null)
  else
    edge ← node.children[c]
    prefix, suffixS, suffixEdge ← longestCommonPrefix(s, edge.label)
    return (edge, prefix, suffixS, suffixEdge)
  
```

Retrieves the outgoing edge in node starting with character `c`

Returns the computed values

Computes the longest common prefix between `s`, the edge's label and the remaining suffixes

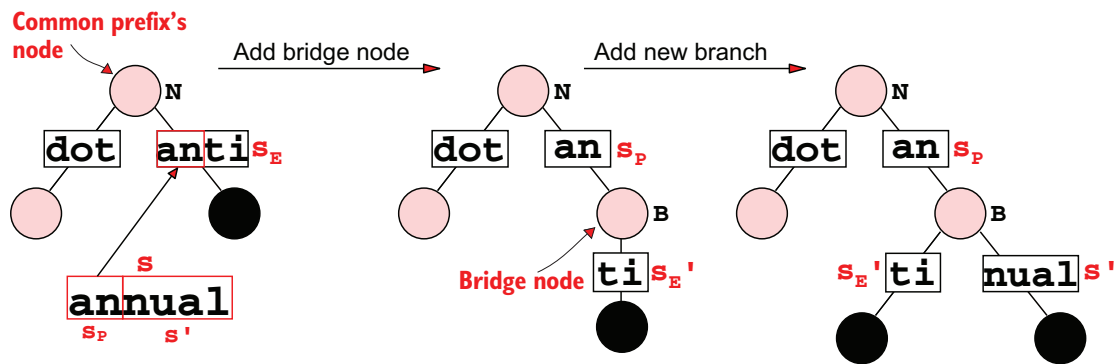


Figure 6.17 Handling case #2 in edge matching while performing an insertion. In this example, we add the word “annual” to a node containing an edge labeled with “anti”. To do so, we insert a bridge node B, linked to N by an edge labeled with the common prefix “an”, and then two new edges leaving B.

To better understand this operation, it might help you to imagine that we decompress the edge to the child into a path, going back to the trie representation with one char per link. Then we traverse this path until we get to the end of the common prefix (to, say, a node B), we add a new branch as a child of B, and finally we compress again the two sub-paths on the two sides of B.

Listing 6.15 Method insert for radix tries

```

Method insert takes a RTNode and the string key s to be
inserted. It returns nothing but has side effects on the trie.
Again, we assume that node is never null. This is a reasonable
assumption if this method is implemented as a private method,
internally called by the trie's API insert method.

function insert(node, s)
    if s == "" then
        node.keyNode ← true
    else
        (edge, commonPrefix, sSuffix, edgeSuffix) ← matchEdge(node, s)
        if edge == null then
            this.children[s[0]] ← new RTEdge(s, new Node(true))
        elif edgeSuffix == "" then
            insert(edge.destination, sSuffix)
        else
            bridge ← new Node(false)
            this.children[s[0]] ← new RTEdge(commonPrefix, bridge)
            bridge.children[edgeSuffix[0]] ←
                new RTEdge(edgeSuffix, edge.destination)
            insert(bridge, sSuffix)

```

Since s is not the empty string, we can see if there is an edge matching it, even partially.

Updates the outgoing edge for this node, with an edge pointing to the bridge node, and labeled with the common prefix

Checks if the string searched is empty. If it is, since this method is implemented using recursion, we know it has traversed the whole path in the trie.

We are at the target node for the target key. We set the current node to a key node to ensure it will store s.

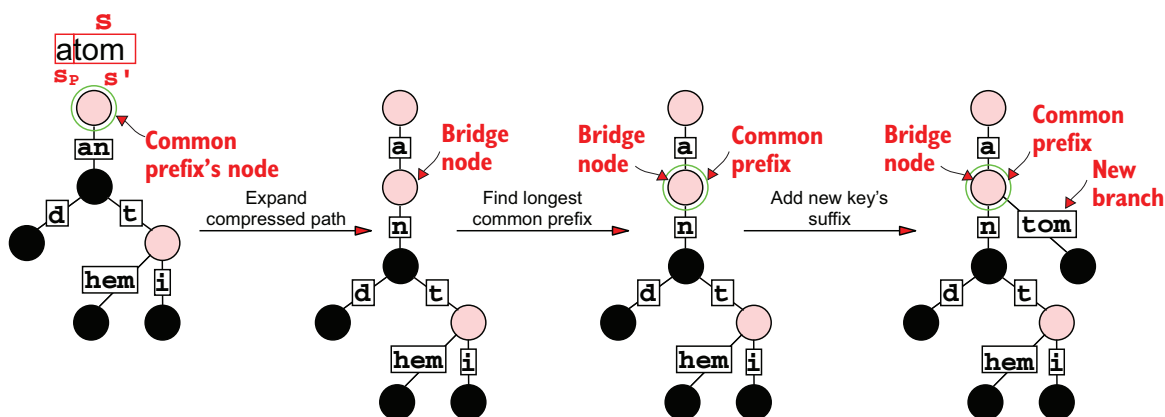
Match case #1. There is an edge whose label is a prefix to s; we just need to traverse the edge.

Match case #4 (figure 6.15). If there isn't any edge sharing a common prefix, not even the first character, with s, then we need to add a new edge, with label s, to a new key node.

Otherwise, we are in match case #2 or #3. There is a common prefix between the edge's label and s, but there are also characters in the edge's label not matching s. Therefore, we need to break down the edge and create a bridge node.

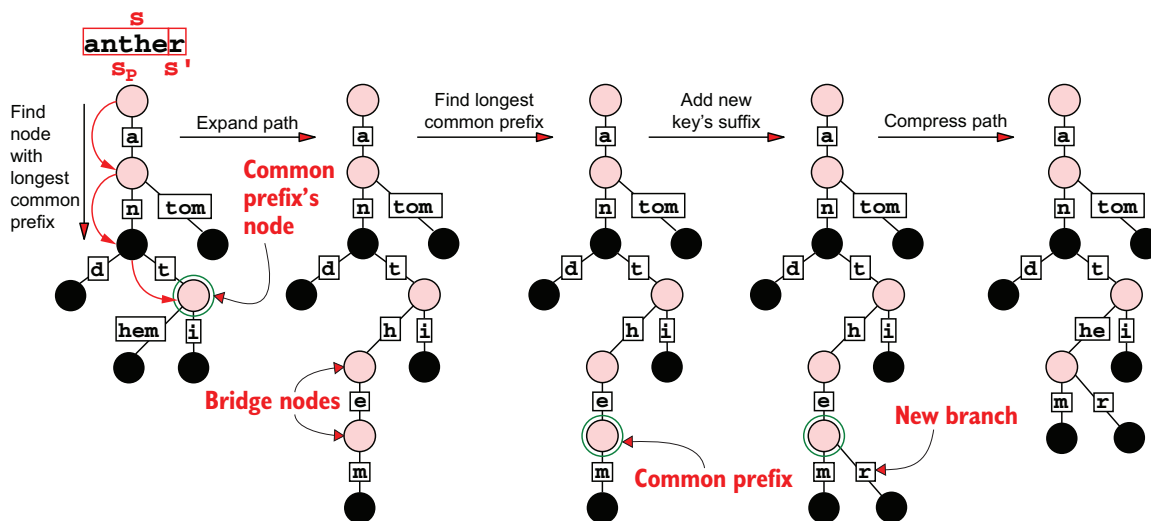
Adds an edge from the bridge node to the former children of the current node: the label will be the original edge's label, stripped of commonPrefix.

Finally, we still need to recursively add the remaining part of the key to the bridge node. If sSuffix is the empty string, this corresponds to match case #3, otherwise to match case #2.

Figure 6.18 An example of method `insert`

Listing 6.15 describes the pseudo-code for the `insert` method, and figures 6.18 and 6.19 show two examples of how this method works on a simplified tree.

The method follows the same high-level logic as the trie's version we saw in listing 6.5. We traverse the tree as far as we can (following the longest path covering a prefix of the string to insert), and then add a new branch for the new key.

Figure 6.19 Another example of method `insert` with path decomposition explained step by step

Traversing the tree becomes more complicated because at each node we need to distinguish between the four different possible results of the edge label matching, and this complexity is reflected in the length of the method. Moreover, when we bump into case #2 or #3, we need to break down an edge and add a bridge node. Adding a new branch, however, becomes easier, because we just need to add a new edge and a single node.

6.3.4 Remove

Like for search, the only changes to the `remove` method, with respect to tries, revolve around the extraction of the common prefix. This is not surprising, because deleting a key can be thought of as a successful search followed by a clean-up of the deleted node.

With `remove`, we don't have to worry about splitting edges or adding bridge nodes. Because we need to find the key first, there must be a path perfectly matching the key to be deleted, in order to be able to complete the operation. We might, however, have the chance to compact the final part of the deleted path, because turning an existing key node into an intermediate node can change the tree structure, introducing a pass-through node (see 6.3.1).

Figure 6.20 shows an example of `remove` in action on a radix tree.

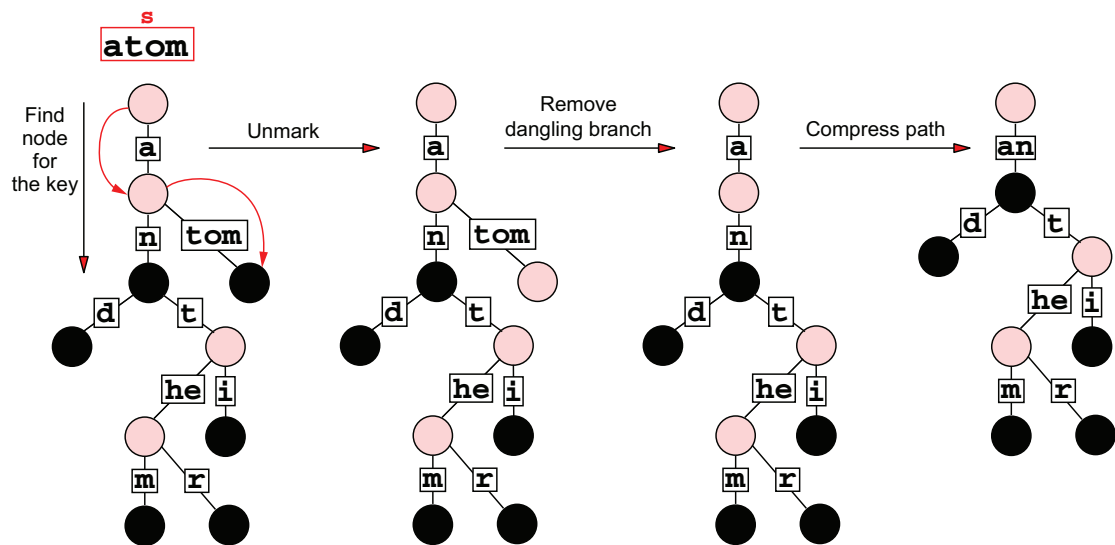


Figure 6.20 Removing the word “atom” from an example radix trie. (1) Find the node at the end of the path for the key to delete. The path must entirely match the key. (2) Unmark the node, making it an intermediate node. If the node is a leaf, it will create a dangling branch. (3) Remove the dangling branch. If the parent of the node removed had only two children, it now became a pass-through node. (4) Compress the path by removing the pass-through node and merging edges.

Besides that, we might have to perform the usual pruning when deleting a key in a leaf. The difference with tries, in this case, is that we will only have to remove a single edge in radix trees.

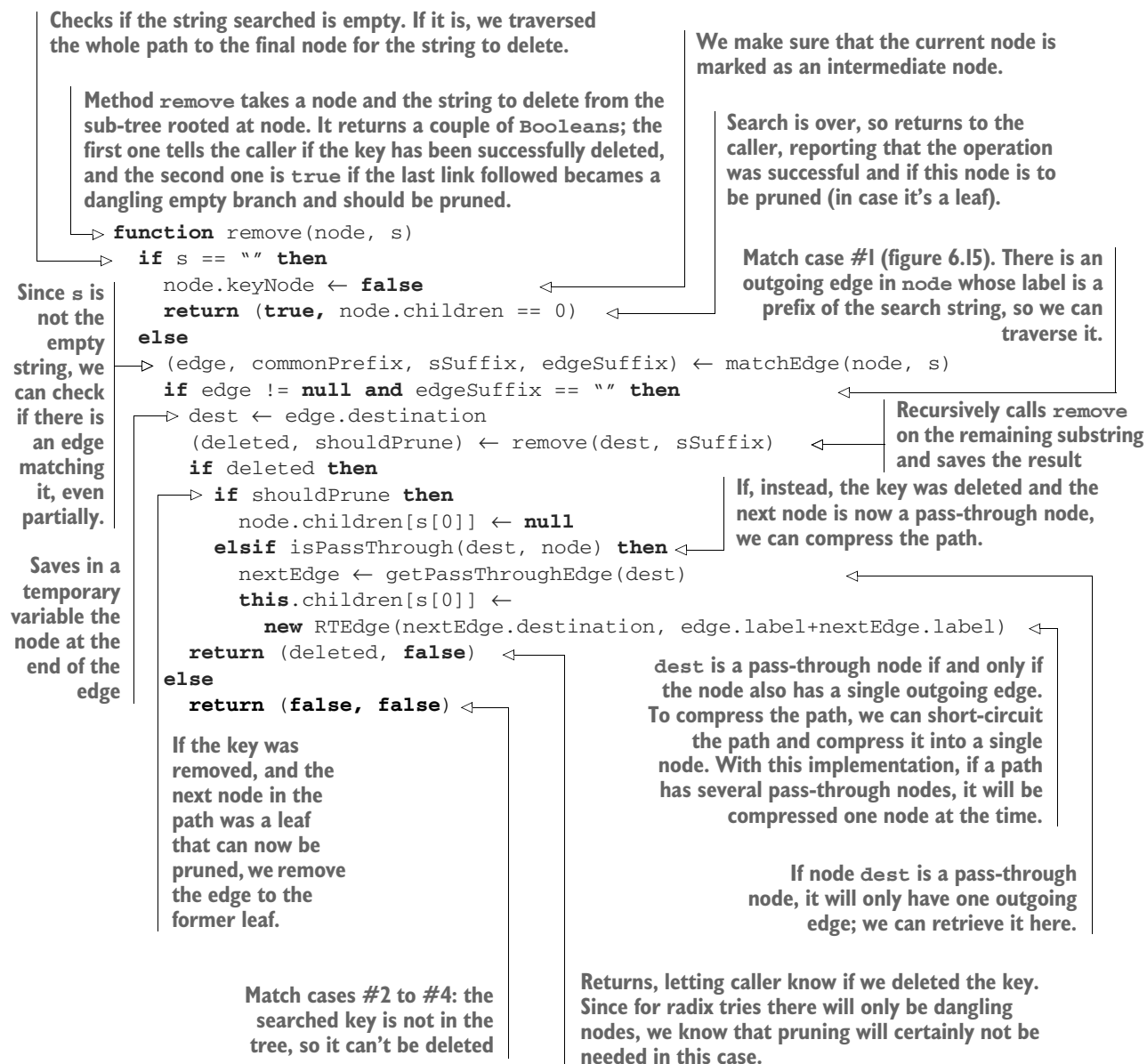
The example in figure 6.20 shows both cases where we have to correct a node's parent. We first remove the key from a leaf, and then, once the node is removed from the tree, its parent becomes a pass-through node, and hence can be removed, allowing us to compress the path from its parent to its only child.

Listing 6.16 shows the pseudo-code for this method, where we use two utility functions.

`isPassThrough` checks if a node is a pass-through node. This only happens when a node is not a key node, it has only one outgoing edge, and even its parent only has one outgoing edge (hence we need to pass the parent too). Implementation is left as an exercise.

Since a pass-through node only has one outgoing edge, its children field will have exactly one entry; `getPassThroughEdge` is a wrapper for retrieving this entry.

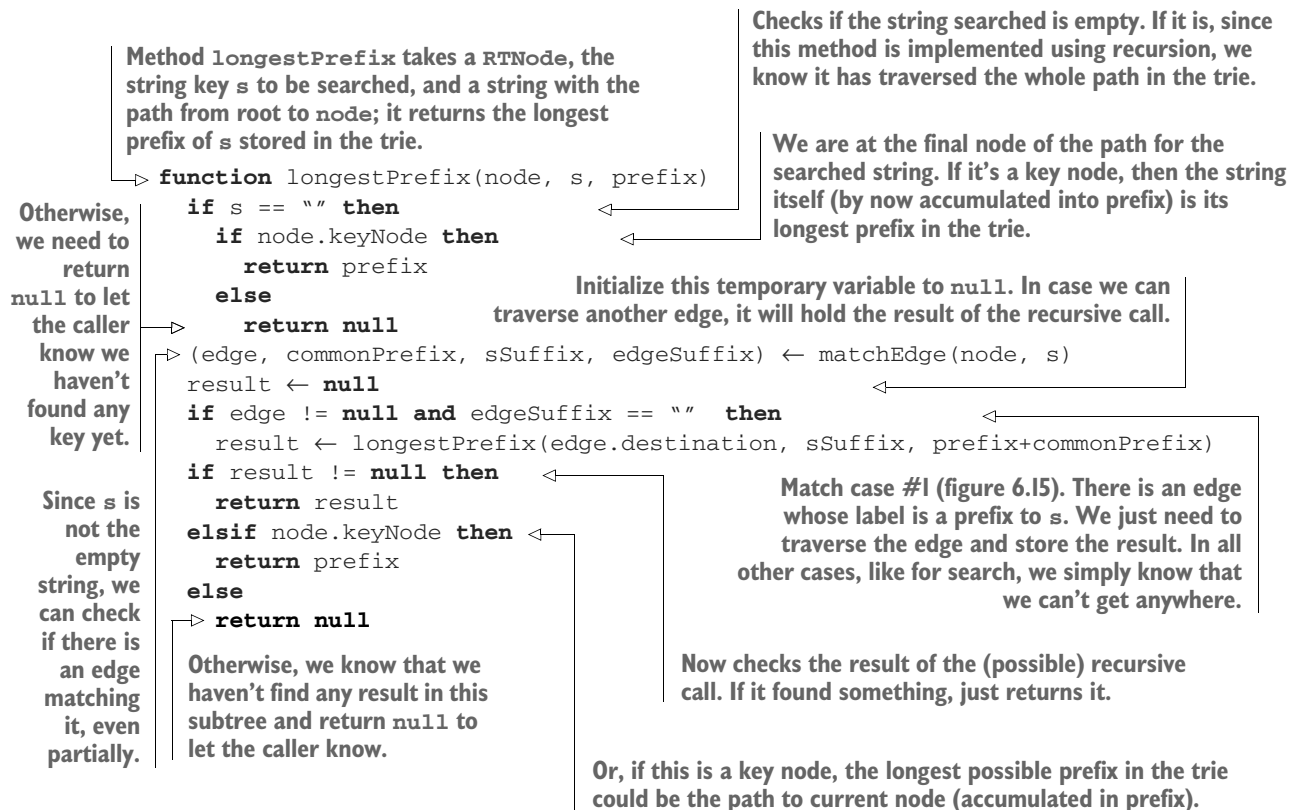
Listing 6.16 Method `remove` for radix tries



6.3.5 Longest common prefix

Porting this method from the trie's version is straightforward. It's just a matter of slightly modifying the search algorithm to take into account the different way we do edge matching. Listing 6.17 describes the pseudo-code for the radix trie's version.

Listing 6.17 Method `longestPrefix` for radix trie



6.3.6 Keys starting with a prefix

For tries, this method leverages search to find the starting point of a full-fledged traversal, retrieving all keys in the subtree rooted at the prefix.

Unfortunately for radix tries, the situation is a bit more complicated, because prefixes that are not stored in the tree can partially match edges. Take, for instance, the tree in figure 6.20, where prefixes such as "a" or "anth" are not stored in the tree. The latter doesn't even have a node at the end of the corresponding path, but still the radix trie contains several words starting with those prefixes.

If we were just looking for nodes that lie at the end of the path for those strings, we would miss all those legit results. We need, instead, to rewrite a special version of the search method for this operation, where we distinguish the edge match case #2, where we have a no-go, and case #3, where, instead, since the string fragment searched is a proper prefix of the last edge in the path, the subtree referenced by the edge will

Match case #3. Although there is no node storing a key at the end of the path for the searched prefix, there would be a pass-through node for it in the uncompressed trie. This means that all prefixes stored in the subtree rooted at `node` will start with `s+edgeSuffix`, and those will be the only strings stored having `s` as a prefix.

Match case #1. There is an edge whose label is a prefix to `s`. We just need to traverse the edge and recurse on the remaining characters in the string.

```

    elif edgeSuffix == "" then
        return searchNodeWithPrefix(edge.destination, sSuffix)
    elif sSuffix == null then
        return edge.destination
    else
        return null

```

Match case #4. There is no path starting with `s` in the (sub)trie rooted at `node`.

This concludes our rundown of the main methods for radix tries, and the discussion on data structures for efficient strings search. To the interested reader who would like to delve further into this subject, we suggest taking a look at suffix trees and suffix arrays, two interesting data structures that are fundamental to fields like bioinformatics, and that are unfortunately out of scope for this chapter.

6.4 Applications

Now that we know two concrete data structures for implementing the ADT String-Container, we can confidently look at applications where they make a difference.

As is usual with data structures, the difference is not about new things that couldn't be done without tries, but rather about doing some operations better or faster than with other DSs.

This is particularly true for tries, as they were specifically designed to improve the running time of string-based queries. As one of the main uses for tries is to implement text-based dictionaries, the touchstone will often be hash tables.

6.4.1 Spell-checker

Time to go back to our main example! We saw in chapter 4 that Bloom filters were used for the first versions of spell-checkers, but after a while they were replaced with more efficient alternatives, such as tries.

The first step to build a spell-checker is, obviously, inserting all the keys from our dictionary (here meant as “English dictionary,” not the data structure!) in a trie.

Then, using the trie for spell check when the feedback we want is just highlighting typos is simple. We just need to perform a search, and if it's a miss, we have a typo.

But suppose, instead, that we would like to also provide suggestions about how we could correct the typo—how can we do that with a trie?

Let's say that the word `w` we are checking has `m` characters, and we can accept suggestions differing by at most `k` characters from `w`: in other words, we want words whose *Levenshtein distance* (also known as edit distance) is at most `k`.

To find those words in a trie, we start traversing the tree from the root, and while traversing it we keep an array of m elements, where the i -th element of this array is the smallest edit distance necessary to match the key corresponding to the current node to the first i characters in our search string.

For each node N , we check the array holding the edit distances:

- If all distances in the array are greater than our maximum tolerance, then we can stop; there's no need to traverse its subtree any further (because the distances can only grow).
- Otherwise, we keep track of the last edit distance (the one for the whole search string), and if it's the best we have found so far, we pair it with current node's key and store it.

When we finish traversing, we will have saved the closest key to our search string and its distance.

Figure 6.22 shows how the algorithm works on a simplified example. It uses a trie, but the same algorithm, with minor changes, can easily be shown and implemented on radix trees. In fact, for this algorithm we are mostly interested in key nodes, not intermediate ones.

The algorithm starts at the root that corresponds to the empty string (because the path that leads to it is also empty). At each step, we have to compare the target word s ("amt", in the example) to the word corresponding to the current node; in particular, we compute the distance between each prefix of s and the word associated to the current node.

So, for the root, the distance between the empty string and the empty prefix of "amt" (which also is the empty string, obviously) is 0 (because they match). The distance between "" and "a" is 1, because we need to add one character to the former to make the latter, and so on.

After computing our vector of distances, we traverse any outgoing edge and repeat the process for the next nodes. In this case, there is only one, associated with the string "a", the concatenation of the labels leading to it. We can build the next row in the table using only the previous row, and compare the last character (the one marking the last edge traversed) to each character in s (note that for the empty string column, the distance will always be the length of the path).

Therefore, the second row in our table will start with a 1, then have 0 in cell [1,1], because both strings start with an 'a'. For the next character in s , there isn't a corresponding char in the node's key (because it's shorter), so we need to add 1 to have its distance, and the same for the last character. In fact, as a double-check, if we consider the prefix "am", the distance to "a" is 1, while for "amt" is 2.

Notice that the cost to compare two strings is always contained in the bottom-right cell, so for "a" and "amt" this distance is 2.

The algorithm goes on traversing all branches, until we get to a point where the cost can't decrease any more (when the path is already as long as the string s or longer, there is no point in going down a branch as soon as we find a key node) or all the

But recently another even more important use case has become popular: bioinformatics, matching sequences of DNA. This is a computationally intensive task, so using the wrong data structures can make it impossible to solve.

When we have to compare just two strings, directly computing the Levenshtein distance is the most effective way to go; however, if we have to compare a single string to n other strings to find the best match, computing n times the Levenshtein distance becomes impractical. The running time would be $O(n * m * M)$, where m is the length of our search string and $O(M)$ is the average length of the n strings in the corpus.

Turns out, we can do much better using a trie. By using the same algorithm shown for spell-checkers (without the threshold-based pruning), computing all the distances will only take time $O(m * N)$, where N is the total number of nodes in the trie, and while the trie construction could take up to $O(n * M)$, it would only happen once at the beginning, and if the rate of lookups is high enough, its cost would be amortized.

In theory N can be $O(n * M)$, as we saw in section 6.2, if no two strings in the corpus share the same prefix. In practice, however, it is likely that N is order of magnitudes smaller than $n * M$, and closer to $O(M)$. Moreover, as we saw in the last sub-section, if we set a threshold for the max tolerance, that is, for the largest difference between two strings, and keep track of the best result we have found, we can prune even more the number of nodes we traverse during search.

6.4.3 String sorting

*Burstersort*²¹ is a cache-efficient sort algorithm that works similarly to MSD (Most Significant Digit) radix sort. However, *burstersort* is cache-efficient and even faster than radix sort!

They both have the same asymptotic running time, $O(n * M)$, which is a theoretical lower bound for sorting n strings of length M , but *burstersort* creates results twice as fast by exploiting locality of reference and better memory distribution.

Going into the details of this algorithm is out of the scope of this chapter, but to give you an idea of how *burstersort* works, it dynamically constructs a trie while the strings are sorted, and uses it to partition them by assigning each string to a bucket (similar to radix sort). The asymptotic cost, as mentioned, is the same as MSD's, because leading characters of each string are inspected once only. The pattern of memory accesses, however, makes better use of cache.

While MSD, prior to the bucket-sorting phase, accesses each string once for each character, *burstersort* accesses each string only once overall. The trie nodes, instead, are accessed randomly.

However, the set of trie nodes is much smaller than the set of strings, so cache is used more wisely.

If the set of string exceeds cache size, *burstersort* becomes considerably faster than any other string sorting algorithms.

²¹Sinha, Ranjan, and Justin Zobel. "Efficient trie-based sorting of large sets of strings." Proceedings of the 26th Australasian computer science conference-Volume 16. Australian Computer Society, Inc., 2003.

6.4.4 T9

T9 was such a big milestone in mobile history that we still (mistakenly) address new mobiles' spell-checkers as T9—though it was abandoned long ago with the advent of smartphones.

The name comes as an abbreviation of “Text on 9 keys,” as the alphabet was (long before mobile phones) divided into groups of three to four characters that would fit into a digital phone numpad.

In the original design for landline phones, every number had to be pressed one to four times to choose every single letter. For instance, 2 had to be pressed once for 'a', twice for 'b', and thrice for 'c'.

Instead, the idea with T9 was that the user would press each key once for each letter in the word, to state that the i -th letter belonged to the group of the k -th button. Then T9 would offer suggestions for possible words made out of those combinations of letters, or even directly provide the right word, if only one possible match was found.

For instance, typing 2-6-3 would select all three letters combinations to form the Cartesian product $[a,b,c] \times [m,n,o] \times [d,e,f]$, and T9 would provide valid English words such as $[and, cod, con, \dots]$.

This was made possible by keeping a trie, and for each key pressed refining the search:

- 1 When keypad button 2 is pressed, we would start traversing the trie, going, in parallel, to the subtrees linked by edges marked with 'a', 'b', 'c' (all three of them would likely be in the trie for any language using the Latin alphabet).
- 2 When the second keypad button is pressed, for each of the three paths that we are currently traversing, T9 checks to see if they have children labeled with 'm', 'n' and 'o', and keeps track of the nodes reached at this second level. Each combination represents a path from the root to a level-2 node. Most likely, not all of the 9 combinations would have a path in the trie: for example, it's unlikely any word will start with “bn”.
- 3 The process continues with the next buttons pressed, until there is no node reachable through the possible paths traversed.

For this specific task, the trie nodes would likely store more than just a Boolean for each key. They would rather store the corpus frequency of that word (for instance, how likely it is that a word is used in English). This way, the most likely result would be returned when more than one result is available: for instance, you would somehow expect that *and* would be preferred over *cod*.

6.4.5 Autocomplete

In the last 10 years or so, we have all become familiar with the autocomplete feature of search boxes. Today we even expect search boxes to provide it by default.

The usual autocomplete workflow is the following: A user on the client side (typically a browser) starts typing in a few letters, and the autocomplete search box shows a few options that start with the characters already typed (Does this ring a bell? Sounds like “all keys with prefix”?). If the set of possible values that could be inserted in the search box was static and small, then it could be transmitted to the client together with the page, cached, and used directly on the client.

This, however, is usually not the case, since the sets of possible values are normally large, and they might change over time, or even be dynamically queried.

So, in real-world applications the client usually sends (asynchronously) a REST request to the server with the characters typed so far.

The application server keeps a trie (or more likely a Patricia tree) with the valid entries, searches for the strings starting with what was inserted so far and a valid prefix, and returns a certain number of entries in this string’s subtree.

When the response comes back to the client, it simply shows the list of results from the server as suggestions.

Autocomplete across the Net

It’s not strictly required that the request is performed through a REST endpoint or that it is sent asynchronously. However, the former indicates a clean design, and without the latter the autocomplete feature would make little sense.

To avoid wasting net bandwidth and server computations, usually the requests to autocomplete are sent every few seconds or when a user stops typing. When a response comes back, the page updates the list of entries shown.

This is still not ideal, because with HTTP/1.1 it is not possible to cancel requests that have already been sent and have reached the server. Moreover, we have no guarantee of the order in which responses come back. Therefore, if more characters are typed or erased before the previous response comes back, we’ll have to keep some kind of versioning for the responses and never replace the results shown if a stale response arrives.

This will be mitigated with HTTP/2, since it introduces cancellable requests, among other cool things.

Summary

- There is a fundamental difference between working with primitive data types like integers or floats and working with strings: while all integers will require the same amount of memory to be stored,²² strings can be of arbitrary length, and hence require arbitrarily many bytes.

²²With exceptions; for instance, the *bignum* integer type in Python can represent arbitrarily large numbers, using a variable number of bytes.

- The length of the string on which a data structure works is an important factor in its asymptotic analysis; this leaves space for further optimizations that are not possible with the simplest data types.
- Tries allow us to more efficiently store and query large sets of strings, assuming many of them share some common prefixes.
- String prefixes are a key factor for this new data structure, which in turn allows us to efficiently perform queries to find strings with a common prefix or, vice versa, given a string find its longest prefix in the dataset.
- Radix tries compress paths, whenever possible, to provide a more compact representation of tries, without having to compromise in terms of complexity or performance.
- From spell-checkers to bioinformatics, many applications and fields manipulating strings can benefit from using tries.